

Universidade Católica do Salvador
Bacharelado em Engenharia de Software

Bruno Dantas de Oliveira

Zaira Dutra Pinto Souza

Gustavo Ricci Ferreira

Matheus Pires Fraga
Caio Caldas Matos de Morais

Segurança e Governança de Dados com uso de LLMs no Desenvolvimento de Software

Salvador

2022

Bruno Dantas de Oliveira
Zaira Dutra Pinto Souza
Gustavo Ricci Ferreira
Matheus Pires Fraga
Caio Caldas Matos de Morais

Segurança e Governança de Dados com uso de LLMs no Desenvolvimento de Software

Trabalho de Conclusão de Curso apresentado à Universidade Católica do Salvador como parte dos requisitos necessários para a obtenção do Título de Engenheiro de Software.
Orientador: Prof. Sheila Tirony

Universidade Católica do Salvador

Salvador
2022

Bruno Dantas de Oliveira
Zaira Dutra Pinto Souza
Gustavo Ricci Ferreira
Matheus Pires Fraga
Caio Caldas Matos de Morais

Segurança e Governança de Dados com uso de LLMs no Desenvolvimento de Software

Trabalho de Conclusão de Curso apresentado à Universidade Católica do Salvador como requisito parcial para a obtenção do título de Engenheiro de Software.

Salvador, 26 de outubro de 2026
Banca Examinadora:

Prof. Sheila Tirony
Universidade Católica do Salvador
Orientador

Prof. Me. Segundo professor
Universidade Católica do Salvador

Prof. Me. Terceiro professor
Universidade Católica do Salvador

empty

empty

empty

Resumo

XXXXX .

Palavras-Chave: SEU TEMA

Abstract

ESCREVA AQUI SEU RESUMO em INGLES

Keywords: 1. Programming teaching. 2. Young people’s lack of IT information. 3. Need of the IT job market.

Lista de figuras

Figura 1 – Diagrama de casos de uso 37
Figura 2 – Arquitetura do mecanismo proposto 40
Figura 3 – Diagrama de sequência do processamento de uma requisição de IA 42
Figura 4 – Fluxograma da pré-filtragem de requisições 43
Figura 5 – Fluxograma da análise e da sanitização 44
Figura 6 – Modelo entidade-relacionamento da configuração 45
Figura 7 – Modelo entidade-relacionamento do registro de auditoria 48
Figura 8 – Diagrama de classes: configuração e análise 49
Figura 9 – Diagrama de classes: registro e interface 50
Figura 10 – Atividade de desenvolvimento por semana 53
Figura 11 – Inicialização do GalileuCLI no terminal 69
Figura 12 – Latência do analisador por tamanho do corpo da requisição 71

Lista de tabelas

Tabela 1 – Questões de pesquisa 18
Tabela 2 – Perfis de usuários e partes interessadas 31
Tabela 3 – Requisitos funcionais 32
Tabela 4 – Requisitos não funcionais 35
Tabela 5 – Resumo dos casos de uso 38
Tabela 6 – Componentes da arquitetura 41
Tabela 8 – Dicionário de dados da entrada de auditoria (campos JSON) 45
Tabela 7 – Dicionário de dados da configuração (arquivo YAML) 47
Tabela 9 – Interface de linha de comando 48
Tabela 10 – Ambiente e ferramentas de desenvolvimento 51
Tabela 11 – Incrementos de desenvolvimento 53
Tabela 12 – Registro de reuniões de acompanhamento e orientações incorporadas 55
Tabela 13 – Registro de decisões técnicas 55
Tabela 14 – Documentação mantida no repositório 58
Tabela 15 – Suítes de teste automatizadas 61
Tabela 20 – Problemas identificados 63
Tabela 16 – Casos de teste de integração e de sistema 65
Tabela 17 – Resultados dos casos de integração (14/10/2026) 66
Tabela 18 – Latência do analisador por tamanho de corpo (ambiente de verificação) 66
Tabela 19 – Modelo de registro de execução de testes 66
Tabela 21 – Matriz de rastreabilidade requisitos–testes 67
Tabela 22 – Ficha técnica do GalileuCLI 68
Tabela 23 – Situação de verificação dos requisitos funcionais 72
Tabela 24 – Situação de verificação dos requisitos não funcionais 72
Tabela 25 – Achados adicionais da revisão do código (*commit* faebae9) 74
Tabela 26 – Atendimento dos objetivos específicos 74
Tabela 27 – Limitações do estudo e do produto 78

Lista de Siglas e Abreviaturas

API	*Application Programming Interface*
BES	Bacharelado em Engenharia de Software
CA	*Certificate Authority* (autoridade certificadora)
CI	*Continuous Integration* (integração contínua)
CLI	*Command-Line Interface* (interface de linha de comando)
HTTP	*Hypertext Transfer Protocol*
HTTPS	*HTTP Secure*
JSON	*JavaScript Object Notation*
LLM	*Large Language Model* (modelo de linguagem de grande escala)
MITM	*Man-in-the-Middle*
MoSCoW	*Must, Should, Could, Won't* (técnica de priorização)
PEM	*Privacy-Enhanced Mail* (formato de certificados e chaves)
RF	Requisito funcional
RNF	Requisito não funcional
SGBD	Sistema gerenciador de banco de dados
TLS	*Transport Layer Security*
TUI	*Terminal User Interface* (interface de terminal)
UC	Caso de uso
YAML	*YAML Ain't Markup Language*

Sumário

1	INTRODUÇÃO	17
1.1	Problema de Pesquisa	18
1.2	Justificativa	18
1.3	Objetivos	19
1.3.1	Objetivo Geral	19
1.3.2	Objetivos Específicos	19
1.4	Delimitação do Escopo	20
1.5	Procedimentos Metodológicos	20
1.6	Estrutura do Trabalho	20
2	FUNDAMENTAÇÃO TEÓRICA	21
2.1	Segurança e Vazamento de Dados no Desenvolvimento Assistido por IA	21
2.2	Modelos de Linguagem de Grande Escala (LLMs) e Riscos de Privacidade	22
2.3	Assistentes de Desenvolvimento (Code Assistants)	22
2.4	Detecção de Dados Sensíveis	23
2.5	Mitigação: Sanitização e Redação de Dados	23
2.6	Arquiteturas Intermediárias de Proteção	24
2.7	Avaliação de Impacto Operacional	24
2.8	Governança de Dados e IA	25
2.9	Trabalhos Relacionados	25
2.9.1	Detecção de Dados Sensíveis	25
2.9.2	Preservação de Privacidade em Modelos de Linguagem	26
2.9.3	Arquiteturas Alternativas de Redução de Exposição	27
2.9.4	Síntese e Posicionamento do Trabalho	27
3	METODOLOGIA	28
3.1	Visão da Pesquisa	28
3.2	Processo de Pesquisa	28
3.2.1	Estratégias e Strings de Busca	28
4	LEVANTAMENTO E ANÁLISE DE REQUISITOS	30
4.1	Abordagem de Levantamento	30
4.2	Identificação dos Usuários	31
4.2.1	Cenário de uso de referência	31

4.3 Requisitos Funcionais . . . 32
4.4 Requisitos Não Funcionais . . . 34
4.5 Restrições, Premissas e Limitações Conhecidas . . . 36
4.6 Casos de Uso . . . 37
4.7 Análise e Priorização . . . 37

5 PROJETO DA SOLUÇÃO . . . 39
5.1 Princípios de Projeto . . . 39
5.2 Arquitetura . . . 39
5.2.1 Visão geral . . . 39
5.2.2 Estilo arquitetural . . . 40
5.2.3 Fluxo de processamento de uma requisição . . . 40
5.2.4 Concorrência e limites operacionais . . . 41
5.2.5 Implantação e segurança . . . 42
5.3 Modelagem dos Dados . . . 43
5.3.1 Decisão sobre o armazenamento . . . 43
5.3.2 Modelo de configuração . . . 44
5.3.3 Modelo de auditoria . . . 44
5.3.4 Mapeamento relacional proposto . . . 46
5.4 Interfaces . . . 48
5.4.1 Interface de linha de comando . . . 48
5.4.2 Interface de terminal (TUI) . . . 49
5.5 Diagrama de Classes . . . 49

6 DESENVOLVIMENTO DA SOLUÇÃO . . . 51
6.1 Preparação do Ambiente . . . 51
6.2 Organização do Repositório . . . 52
6.3 Implementação por Incrementos . . . 52
6.4 Reuniões de Acompanhamento . . . 55
6.5 Registro das Decisões . . . 55
6.6 Atualizações da Documentação . . . 57

7 TESTES E VALIDAÇÃO . . . 59
7.1 Planejamento dos Testes . . . 59
7.1.1 Objetivos e níveis . . . 59
7.1.2 Ambiente, dados e critérios . . . 59
7.2 Casos de Teste . . . 60
7.2.1 Testes automatizados existentes . . . 60
7.2.2 Casos de teste de integração e de sistema . . . 60
7.3 Execução e Registro de Evidências . . . 60

7.3.1 Procedimento . . . 60
7.3.2 Resultados dos testes automatizados . . . 61
7.3.3 Resultados dos testes de integração . . . 62
7.3.4 Resultados de desempenho . . . 62
7.3.5 Evidências adicionais documentadas no repositório . . . 62
7.3.6 Modelo de registro de evidências . . . 62
7.4 Problemas Identificados e Correções . . . 63
7.5 Rastreabilidade entre Requisitos e Testes . . . 64
7.6 Síntese . . . 64

8 RESULTADOS E DISCUSSÃO . . . 68
8.1 Apresentação do Produto . . . 68
8.1.1 Como o produto é usado . . . 69
8.2 Resultados Obtidos . . . 70
8.2.1 Detecção e redação . . . 70
8.2.2 Desempenho . . . 71
8.2.3 Situação dos requisitos . . . 71
8.2.4 Defeitos e achados adicionais . . . 73
8.2.5 Validação com usuários . . . 73
8.3 Comparação com os Objetivos . . . 73
8.4 Análise Crítica e Discussão . . . 75
8.4.1 O que os resultados de detecção sustentam e o que não sustentam 75
8.4.2 Comparação com a literatura . . . 76
8.4.3 Custo operacional . . . 77
8.4.4 Decisões de projeto e suas consequências . . . 77
8.4.5 Governança e auditabilidade . . . 77
8.4.6 Ameaças à validade . . . 78
8.5 Limitações . . . 78

9 CONCLUSÕES . . . 80
9.1 Retomada do Problema . . . 80
9.2 Avaliação dos Objetivos . . . 80
9.3 Contribuições . . . 81
9.4 Limitações . . . 81
9.5 Recomendações para Trabalhos Futuros . . . 81
9.6 Considerações Finais . . . 82

REFERÊNCIAS . . . 83

A APÊNDICE . . . 84

1 Introdução

Assistentes de programação baseados em modelos de linguagem de grande escala (*Large Language Models*, LLMs) tornaram-se parte do cotidiano do desenvolvimento de software. Integrados a editores, a ambientes de desenvolvimento e à linha de comando, eles completam código, sugerem refatorações e respondem a perguntas sobre o projeto. Para isso precisam de contexto: a cada interação, trechos de código, arquivos abertos, mensagens de erro e, com frequência, arquivos de configuração são enviados a provedores externos de inteligência artificial, fora do domínio de controle da organização que utiliza a ferramenta.

Esse fluxo cria uma superfície de exposição de dados que os controles de segurança tradicionais tratam mal. O dado sensível não é subtraído por um atacante: é enviado, de forma legítima e muitas vezes inadvertida, pela própria ferramenta autorizada a um destino também autorizado (Capítulo 2). Credenciais de acesso, chaves de API, *tokens* e identificadores internos que acompanham o contexto passam, assim, a residir na infraestrutura de terceiros. Casos reais indicam a dimensão do problema: em 2023, engenheiros da *Samsung* submeteram código-fonte proprietário a um assistente de IA generativa, o que resultou em três incidentes de vazamento em cerca de um mês e na proibição da ferramenta pela empresa; e relatórios recentes de monitoramento de segredos apontam volume crescente de credenciais expostas em repositórios públicos, inclusive em arquivos de configuração ligados a ferramentas de IA.

Diante desse quadro, as organizações tendem a oscilar entre proibir o uso das ferramentas, abrindo mão de seus ganhos de produtividade, e aceitar o risco. Entre os dois extremos há espaço para mecanismos técnicos que permitam usar o assistente sem que o segredo chegue ao provedor. A literatura revisada neste trabalho oferece peças desse mecanismo, mas não o conjunto: há métodos de detecção de segredos e de dados pessoais avaliados sobre artefatos estáticos, técnicas de preservação de privacidade aplicadas ao treinamento ou à arquitetura interna do agente, e assistentes executados localmente que dispensam o provedor externo. Não foi identificado, entre os trabalhos analisados, um que combine a interceptação do tráfego em tempo real, a independência em relação à ferramenta e ao provedor, e a avaliação do impacto operacional da camada de proteção (Capítulo 2). É nessa lacuna que este trabalho se situa.

1.1 Problema de Pesquisa

O problema que orienta este trabalho pode ser formulado da seguinte maneira: *como impedir que dados sensíveis presentes no contexto enviado por ferramentas de desenvolvimento assistidas por LLM cheguem a provedores externos de IA, sem exigir a substituição da ferramenta nem do provedor e sem degradar de forma perceptível a experiência do desenvolvedor?*

Esse problema se desdobra nas questões de pesquisa (*Research Questions*, RQs) da Tabela 1, cada uma associada a uma das cadeias de busca da revisão sistemática descrita no Capítulo 3. As quatro primeiras são respondidas pela revisão da literatura (Capítulo 2); a quinta é respondida, também, pela verificação experimental do mecanismo proposto (Capítulos 7 e 8).

Tabela 1 – Questões de pesquisa

ID	**Questão**	**Cadeia**	**Respondida em**
RQ1	Que riscos de exposição de dados, código e privacidade o uso de assistentes de programação baseados em LLM introduz?	A	Cap. 2
RQ2	Que técnicas de sanitização, redação e anonimização são empregadas para remover ou substituir informação sensível antes do envio ao modelo?	B	Cap. 2
RQ3	Que arquiteturas intermediárias (*proxies*, *gateways*, *middlewares*) são utilizadas para prevenir a perda de dados?	C	Cap. 2
RQ4	Que métodos permitem detectar credenciais, segredos e dados pessoais em código e em ambientes de desenvolvimento?	D	Cap. 2
RQ5	Qual é o impacto operacional (latência, utilidade, produtividade) de introduzir uma camada de proteção nesse fluxo?	E	Cap. 2, 7 e 8

Fonte: Elaborado pela equipe (2026), a partir das cadeias de busca da Seção 3.2.1.

1.2 Justificativa

Do ponto de vista prático, o uso de assistentes de código já é decisão consolidada em muitas equipes, de modo que a alternativa de eliminar o provedor externo (como em assistentes locais) nem sempre está disponível. Uma camada de proteção que atue sobre o tráfego, sem alterar a ferramenta do desenvolvedor, reduz o custo de adoção e permite que a política de dados da organização seja aplicada, e não apenas declarada.

Do ponto de vista científico, a revisão da literatura mostra que as abordagens existentes avaliam a detecção sobre artefatos estáticos ou atuam no treinamento do

modelo, e que a latência introduzida por uma camada de proteção não é avaliada nos estudos analisados. O trabalho contribui ao propor um mecanismo que opera sobre o tráfego em trânsito e ao reportar medições do seu custo operacional.

Do ponto de vista social e de governança, o mecanismo gera uma trilha de auditoria que registra o que foi detectado sem armazenar o conteúdo das requisições, o que oferece à organização evidência para acompanhar o uso das ferramentas de IA. Por ser distribuído como *software* de código aberto, o resultado pode ser examinado, reproduzido e evoluído por terceiros.

1.3 Objetivos

1.3.1 Objetivo Geral

Projetar, implementar e avaliar um mecanismo de *proxy* intermediário, independente da ferramenta e do provedor, capaz de detectar e redigir dados sensíveis no tráfego HTTP enviado por ferramentas de desenvolvimento assistidas por LLM a provedores externos de IA, e avaliar o seu impacto operacional.

1.3.2 Objetivos Específicos

OE1. Identificar, por meio de revisão sistemática da literatura, os riscos de exposição de dados no uso de assistentes de programação baseados em LLM e as técnicas existentes de detecção, sanitização e proteção por arquiteturas intermediárias, caracterizando as lacunas a que o trabalho responde.

OE2. Levantar e especificar os requisitos funcionais e não funcionais do mecanismo, identificando seus usuários e definindo critérios de aceitação mensuráveis sempre que possível.

OE3. Projetar a arquitetura, o modelo de dados e as interfaces do mecanismo, documentando-os em diagramas.

OE4. Implementar o mecanismo de forma incremental e em código aberto, registrando as decisões técnicas e mantendo a documentação atualizada.

OE5. Verificar a eficácia da detecção e da redação, considerando a detecção dos formatos de credencial suportados, a ausência de falsos positivos nos casos de teste e a integridade das requisições encaminhadas.

OE6. Avaliar o impacto operacional do mecanismo, medindo a latência e a vazão da análise para corpos de requisição de diferentes tamanhos.

1.4 Delimitação do Escopo

O trabalho trata dos dados que saem da máquina do desenvolvedor em requisições HTTP(S) dirigidas a provedores de IA. A detecção baseia-se em padrões (expressões regulares e valores literais) voltados principalmente a credenciais de formato conhecido, configuráveis pelo usuário. Ficam fora do escopo: o treinamento ou o ajuste fino de modelos; a inspeção das respostas dos provedores; a classificação semântica de conteúdo e a detecção de código proprietário por similaridade; a avaliação da qualidade das respostas do LLM após a redação; e a demonstração de conformidade com normas específicas de proteção de dados. As consequências dessas delimitações são discutidas no Capítulo 8.

1.5 Procedimentos Metodológicos

Trata-se de pesquisa aplicada, de natureza tecnológica, composta por três frentes. A primeira é uma revisão sistemática da literatura, conduzida com cinco cadeias de busca na base IEEE Xplore e triada com apoio da ferramenta Parsifal, que resultou em oito trabalhos incluídos entre quinze trabalhos únicos analisados (Capítulos 3 e 2). A segunda é o desenvolvimento incremental do mecanismo, precedido do levantamento de requisitos e do projeto da solução e acompanhado do registro das decisões técnicas (Capítulos 4 a 6). A terceira é a verificação do mecanismo por testes automatizados, de integração e de desempenho (Capítulo 7), cujos resultados são analisados à luz dos objetivos no Capítulo 8.

1.6 Estrutura do Trabalho

Além desta introdução, o trabalho está organizado em oito capítulos. O Capítulo 2 apresenta a fundamentação teórica e os trabalhos relacionados. O Capítulo 3 descreve a metodologia da revisão. Os Capítulos 4, 5 e 6 documentam o levantamento de requisitos, o projeto e o desenvolvimento da solução. O Capítulo 7 reúne o planejamento, a execução e os resultados dos testes. O Capítulo 8 apresenta o produto, os resultados e sua discussão, a comparação com os objetivos e as limitações. O Capítulo 9 apresenta as conclusões, as contribuições e as recomendações para trabalhos futuros.

2 Fundamentação Teórica

Esta seção apresenta uma breve fundamentação téorica sobre.....

2.1 Segurança e Vazamento de Dados no Desenvolvimento Assistido por IA

A adoção de ferramentas de desenvolvimento assistidas por modelos de linguagem de grande escala introduz uma nova superfície de exposição de dados: cada interação entre o desenvolvedor e o assistente, seja um trecho de código, uma mensagem de commit ou o contexto de um arquivo aberto no editor é enviada a um provedor externo de inteligência artificial para ser processada. Esse fluxo caracteriza o que a literatura denomina *data leakage*, definido como a transmissão não autorizada ou não intencional de informação sensível para fora do domínio de controle da organização, e sua variação mais específica ao contexto deste trabalho, o *code leakage*, que se refere à exposição de código-fonte proprietário, segredos de aplicação e credenciais de acesso durante o uso de ferramentas externas de desenvolvimento.

Diferentemente de vazamentos causados por falhas de configuração de infraestrutura ou ataques direcionados, a exposição de dados em ferramentas assistidas por LLM ocorre como efeito colateral do uso legítimo da ferramenta: o dado sensível não é roubado, é enviado voluntariamente pelo próprio usuário como parte do fluxo normal de trabalho, o que torna esse risco mais difícil de mitigar por controles de segurança perimetrais tradicionais, uma vez que o tráfego se origina de dentro do ambiente de desenvolvimento e é direcionado a um destino legítimo e autorizado pela organização.

Como o ocorrido na empresa *Samsung* em 2023, onde os seus engenheiros usaram o *ChatGPT* para avaliar o código-fonte e otimizar as sequências de testes para identificar falhas nos chips que estavam projetando. Causando três situações de vazamento de dados no intervalo de um mês, culminando na proibição do uso da ferramenta pelos funcionários da empresa e na discussão de formas seguras de utilização.

Em 2026, a ferramenta de detecção de segredos *GitGuardian*, da plataforma *GitHub*. Registrou aproximadamente 29 milhões de chaves, tokens e credenciais expostas. E mais de 24.000 segredos únicos em arquivos de configuração MCP no GitHub público. Reforçando o fato de que o uso irrestrito das ferramentas trazem riscos a segurança.

2.2 Modelos de Linguagem de Grande Escala (LLMs) e Riscos de Privacidade

Modelos de linguagem de grande escala (LLMs) são sistemas de aprendizado profundo treinados sobre grandes volumes de texto, capazes de gerar e interpretar linguagem natural e código-fonte a partir de um contexto (*prompt*) fornecido pelo usuário. No cenário de ferramentas de desenvolvimento assistidas por IA, esse contexto tipicamente inclui não apenas a instrução explícita do usuário, mas também trechos do código-fonte do projeto, estrutura de arquivos e, em alguns casos, o histórico de interações anteriores, dados que trafegam até a infraestrutura do provedor do modelo, frequentemente hospedada por terceiros e fora do domínio de controle direto da organização que utiliza a ferramenta.

Xu et al. (2025) sistematizam os principais vetores de risco à privacidade decorrentes desse fluxo, categorizando-os em riscos durante o treinamento do modelo, quando dados sensíveis podem ser memorizados e posteriormente reproduzidos, e riscos durante a inferência, quando o próprio conteúdo do prompt enviado pelo usuário constitui a superfície de exposição. Os autores destacam que técnicas de computação segura, como criptografia homomórfica, ambientes de execução confiável (TEE) e computação multipartidária segura (MPC), têm sido propostas para mitigar esses riscos, ainda que com custos computacionais e de latência relevantes para cenários de uso interativo. Em linha semelhante, Zhang et al. (2024) argumentam que, em arquiteturas de agentes que utilizam ferramentas externas, o risco de exposição se amplia porque o agente frequentemente repassa dados do usuário não apenas ao modelo de linguagem, mas também às ferramentas que ele orquestra como motores de busca, interpretadores de código e integrações com sistemas de terceiros, multiplicando os pontos de possível vazamento.

2.3 Assistentes de Desenvolvimento (Code Assistants)

Assistentes de desenvolvimento baseados em IA (*code assistants*) são ferramentas integradas ao ambiente de desenvolvimento (IDE) ou à linha de comando que utilizam LLMs para auxiliar tarefas como completude de código, geração de funções, refatoração e revisão. Para operar de forma útil, esses assistentes precisam de acesso ao contexto do projeto, arquivos abertos, estrutura de diretórios, histórico de versionamento, e esse contexto é, em arquiteturas convencionais baseadas em provedores de nuvem, transmitido integralmente ao provedor externo a cada interação.

Atrassi, Idrissi e Benkaouz (2025) ilustram uma abordagem arquitetural alternativa a esse modelo com o *CodeWisp*, um assistente de código que opera localmente, utilizando geração aumentada por recuperação (RAG) com indexação semântica guiada

por árvore de sintaxe abstrata (AST) para localizar e fornecer apenas os trechos de código relevantes ao contexto da tarefa, sem depender de um LLM hospedado externamente. Embora a proposta demonstre viabilidade técnica, com recall de 0,96 e *Mean Reciprocal Rank* de 0,95 em tarefas de completude, ela pressupõe a substituição do provedor externo por um modelo local, o que não se aplica a cenários em que o uso de LLMs de terceiros (como os grandes provedores comerciais) já está consolidado como decisão de produto ou de negócio da organização.

2.4 Detecção de Dados Sensíveis

A detecção de dados sensíveis é o processo de identificar, em um fluxo de texto ou código, elementos que representam informação confidencial, credenciais de acesso, chaves de API, dados pessoalmente identificáveis (PII) ou segredos de aplicação, antes que esse conteúdo seja transmitido a um destino não confiável. As abordagens de detecção evoluíram de técnicas baseadas puramente em expressões regulares e análise de entropia, que apresentam alta taxa de falsos positivos por não considerarem o contexto semântico do conteúdo analisado, para abordagens híbridas e baseadas em aprendizado de máquina.

Rahman et al. (2025) demonstram essa evolução ao combinar extração de candidatos via regex com classificação por modelos de linguagem ajustados (LLaMA-3.1 8B e Mistral-7B), alcançando F1-score de 0,9852 na tarefa de classificação binária de segredos em código-fonte. Tsebak, Banakh e Piskozub (2025) seguem abordagem semelhante ao integrar um modelo transformer contextual (CodeBERT) a um classificador de gradient boosting (XGBoost), reportando redução significativa de falsos positivos frente a métodos tradicionais de entropia. No contexto de dados pessoais em vez de segredos técnicos, Sakapertana et al. (2025) comparam algoritmos clássicos de aprendizado supervisionado, SVM e Naive Bayes para detecção de PII em respostas de API REST, com SVM atingindo 80,96% de acurácia, enquanto Khan e Rajagopalan (2025) propõem o uso de modelos generativos em regime *zero-shot*, capazes de se adaptar a novos critérios de sensibilidade sem necessidade de reentreinamento, uma característica relevante para cenários em que a definição do que constitui ”dado sensível”pode variar entre organizações.

2.5 Mitigação: Sanitização e Redação de Dados

Uma vez detectado, o dado sensível precisa ser tratado antes de prosseguir ao destino externo. A literatura de privacidade descreve um espectro de técnicas de mitigação: a *redação* (*redaction*), que consiste na remoção ou ocultação total do dado sensível identificado; a *sanitização* (*sanitization*), termo mais amplo que engloba qual-

quer transformação do conteúdo para reduzir seu risco antes da transmissão; e a *anonimização* e *pseudonimização*, que substituem o valor original por um identificador não reversível ou reversível apenas sob controle específico, respectivamente, permitindo preservar a utilidade estrutural do dado sem expor seu valor real.

Zhang et al. (2024) aplicam esse princípio no nível arquitetural, propondo um mecanismo de embaralhamento (*shuffling*) de atributos combinado a criptografia homomórfica para que agentes de LLM operem sobre dados protegidos sem acesso ao valor original do usuário. Pan e Wu (2025) tratam a mitigação em outro ponto do ciclo de vida do modelo, aplicando ruído customizado a tokens privados durante o processo de ajuste fino (*fine-tuning*) em cenários de aprendizado federado, de forma seletiva, isto é, calibrando o nível de proteção conforme a sensibilidade de cada token, em vez de aplicar uma transformação uniforme a todo o conteúdo. Essa noção de seletividade é particularmente relevante para o mecanismo proposto neste trabalho, pois a redação indiscriminada de todo o tráfego comprometeria a utilidade do assistente de código para o desenvolvedor.

2.6 Arquiteturas Intermediárias de Proteção

Arquiteturas de proteção posicionadas entre o cliente e o serviço externo, como *proxies* reversos, *API gateways* e *middlewares*, constituem um padrão consolidado em segurança de aplicações para implementar pontos de aplicação de política (*policy enforcement points*), nos quais regras de segurança, autenticação e inspeção de conteúdo podem ser aplicadas de forma centralizada e transparente às aplicações que consomem o serviço, sem exigir alterações no código-cliente.

2.7 Avaliação de Impacto Operacional

A introdução de qualquer mecanismo de inspeção ou transformação de tráfego entre o cliente e o provedor externo tem potencial de impactar a experiência de uso da ferramenta, seja por meio do aumento de latência introduzido pelo processamento adicional, seja pela redução de utilidade quando o mecanismo de proteção remove ou distorce informação relevante ao funcionamento correto do assistente. A avaliação desse impacto operacional, portanto, não pode se limitar a métricas de eficácia da detecção como precisão e revocação, devendo incorporar também métricas de eficiência, como tempo de resposta e taxa de falsos positivos, que representam o custo prático de se utilizar o mecanismo de proteção no dia a dia do desenvolvedor.

Os trabalhos revisados na Seção 3.4 fornecem parâmetros de referência para essa avaliação: Rahman et al. (2025) e Tsebak, Banakh e Piskozub (2025) reportam F1-score e taxa de falsos positivos como métricas centrais de eficácia de seus classifica-

dores, enquanto a redução de falsos positivos é tratada por Tsebak, Banakh e Piskozub (2025) como requisito prático para viabilizar a adoção da ferramenta em fluxos de trabalho reais, um paralelo direto ao critério de usabilidade que este trabalho pretende avaliar para o mecanismo de proxy proposto. Nenhum dos trabalhos revisados, no entanto, avalia explicitamente a métrica de latência introduzida pelo mecanismo de proteção, o que reforça a avaliação de impacto operacional como uma contribuição relevante deste trabalho frente à literatura atual.

2.8 Governança de Dados e IA

Governança de dados refere-se ao conjunto de políticas, papéis, processos e controles que definem como os dados de uma organização são geridos, protegidos e utilizados ao longo de seu ciclo de vida. No contexto de adoção de ferramentas de IA generativa no desenvolvimento de software, a governança se materializa na definição de quais categorias de dados podem ou não ser enviadas a provedores externos, em quais condições, e sob qual mecanismo de controle e auditoria essa política é efetivamente aplicada, e não apenas declarada.

Xu et al. (2025) situam esse desafio ao observar que técnicas de computação segura, embora tecnicamente eficazes, ainda carecem de mecanismos maduros de integração com políticas de governança organizacionais em escala, o que limita sua adoção prática fora de ambientes controlados de pesquisa. Pan e Wu (2025) ilustram, por outro lado, como decisões de governança podem ser incorporadas diretamente ao mecanismo técnico de proteção, ao permitir que o nível de proteção aplicado a cada dado seja calibrado conforme sua sensibilidade, uma abordagem que aproxima a implementação técnica da política de governança que a motiva, princípio que orienta o desenho do mecanismo de detecção e redação proposto neste trabalho.

2.9 Trabalhos Relacionados

Esta seção apresenta uma análise comparativa dos trabalhos mais próximos ao tema desta pesquisa, identificados a partir da revisão sistemática conduzida com as Strings A a E (3.2.1). Os trabalhos foram organizados em três frentes: detecção de dados sensíveis, técnicas de preservação de privacidade em modelos de linguagem, e arquiteturas alternativas para redução de exposição de dados.

2.9.1 Detecção de Dados Sensíveis

Um conjunto expressivo dos trabalhos analisados concentra-se na detecção de segredos, credenciais e informações pessoalmente identificáveis (PII) em artefatos de

software. Rahman et al. (2025) propõem uma abordagem híbrida que combina extração de candidatos via expressões regulares com classificação por meio de modelos de linguagem ajustados (LLaMA-3.1 8B e Mistral-7B), alcançando F1-score de 0,9852 na classificação binária e acurácia de 98,2% na classificação multiclasse, superando abordagens baseadas exclusivamente em regex. Em linha semelhante, Tsebak, Banakh e Piskozub (2025) combinam um modelo transformer contextual (CodeBERT) com um classificador de gradient boosting (XGBoost) para detecção de segredos em sistemas de controle de versão, reduzindo significativamente a taxa de falsos positivos frente a métodos tradicionais baseados em entropia.

No contexto de APIs REST, Sakapertana et al. (2025) comparam algoritmos clássicos de aprendizado de máquina, Support Vector Machines e Naive Bayes para detecção de PII em respostas de API, obtendo acurácia de 80,96% com SVM. Já Khan e Rajagopalan (2025) exploram modelos generativos em regime *zero-shot* para classificação de conteúdo sensível, destacando a vantagem de adaptação dinâmica a novos critérios de sensibilidade sem a necessidade de reentreinamento.

Observa-se que, embora estes trabalhos apresentem resultados robustos em precisão de detecção, todos são avaliados sobre artefatos estáticos, código-fonte versionado, respostas de API previamente coletadas ou documentos e não sobre tráfego de rede interceptado em tempo real entre uma ferramenta de desenvolvimento e um provedor externo de inteligência artificial.

2.9.2 Preservação de Privacidade em Modelos de Linguagem

Uma segunda frente de pesquisa trata da preservação de privacidade no próprio ciclo de vida do modelo de linguagem. Zhang et al. (2024) propõem o *PrivacyAsst*, um framework voltado a agentes de LLM que utilizam ferramentas externas, empregando criptografia homomórfica e um mecanismo de embaralhamento de atributos para ocultar entradas individuais dos usuários. Pan e Wu (2025) tratam do problema em outro ponto do ciclo de vida do modelo, propondo um mecanismo de aprendizado federado seletivo que adiciona ruído customizado a tokens privados durante o ajuste fino (*fine-tuning*) de LLMs, mitigando a exposição de dados de domínio específico. Xu et al. (2025) sistematizam, em um trabalho de revisão, as principais técnicas de computação segura aplicadas a LLMs, criptografia homomórfica, ambientes de execução confiável (TEE) e computação multipartidária segura (MPC), consolidando o estado da arte e apontando a escalabilidade como um desafio ainda em aberto.

Esses trabalhos convergem para soluções aplicadas ao *treinamento* ou à *arquitetura interna do agente*, e não a uma camada de interceptação independente, capaz de operar de forma agnóstica em relação à ferramenta de desenvolvimento e ao provedor de IA utilizados.

2.9.3 Arquiteturas Alternativas de Redução de Exposição

Uma abordagem distinta é adotada por Atrassi, Idrissi e Benkaouz (2025), que apresentam o *CodeWisp*, um assistente de código local baseado em geração aumentada por recuperação (RAG), com indexação semântica guiada por árvore de sintaxe abstrata (AST). O sistema alcança recall de 0,96 e *Mean Reciprocal Rank* de 0,95 em tarefas de completude de código, evitando a necessidade de envio de dados a provedores de LLM hospedados em nuvem. Trata-se, no entanto, de uma estratégia de *eliminação* do provedor externo, e não de *sanitização* do tráfego já direcionado a ele, uma premissa que não se aplica a cenários em que o uso de provedores externos de IA é uma decisão de negócio já estabelecida e não passível de substituição imediata por um modelo local.

2.9.4 Síntese e Posicionamento do Trabalho

A análise dos oito trabalhos revisados evidencia uma lacuna: nenhum deles combina, simultaneamente, (i) a interceptação de tráfego HTTP em tempo real entre ferramentas de desenvolvimento assistidas por LLM e provedores externos de IA, (ii) o agnosticismo em relação à ferramenta cliente e ao provedor de destino, e (iii) a avaliação sistemática do impacto operacional,latência e utilidade da camada de proteção introduzida. Os trabalhos voltados à detecção (Rahman et al. (2025); Sakapertana et al. (2025); Khan e Rajagopalan (2025); Tsebak, Banakh e Piskozub (2025)) concentram-se em artefatos estáticos; os voltados à preservação de privacidade em LLM (Zhang et al. (2024); Pan e Wu (2025); Xu et al. (2025)) atuam no treinamento ou na arquitetura do agente; e a alternativa arquitetural identificada (Atrassi, Idrissi e Benkaouz (2025)) resolve o problema por substituição do provedor, não por interceptação. É nesse espaço que se posiciona o mecanismo de proxy reverso proposto neste trabalho, ao aplicar técnicas de detecção de dados sensíveis diretamente sobre o tráfego HTTP em trânsito, de forma transparente à ferramenta de desenvolvimento e ao provedor de IA utilizados.

3 Metodologia

Esta seção descreve os ḿetodos de pesquisa utilizado como base para realização do estudo.

3.1 Visão da Pesquisa

A pesquisa exposta teve como objetivo promover um estudo .....

3.2 Processo de Pesquisa

3.2.1 Estratégias e Strings de Busca

Para a condução da revisão sistemática e identificação dos trabalhos relacionados, foram estruturadas cinco strings de busca (A, B, C, D e E). Cada consulta foi desenhada com um objetivo específico e vinculada às Questões de Pesquisa (RQs) estabelecidas para este trabalho.

String A: Riscos em assistentes de programação
Objetivo: Identificar riscos de exposição de dados, código proprietário, informações confidenciais e problemas de privacidade no uso de assistentes de programação.

```
("large language model" OR LLM OR "generative AI")
AND
("code assistant" OR "coding assistant" OR "AI coding assistant"
OR "AI-assisted programming" OR "AI-assisted development")
AND
("data leakage" OR "code leakage" OR "privacy leakage")
```

String B: Sanitização e redação
Objetivo: Identificar técnicas utilizadas para remover, substituir, anonimizar ou bloquear informações sensíveis antes do envio ao modelo.

```
("large language model" OR LLM OR "generative AI")
AND
("prompt sanitization" OR "data redaction" OR "input filtering"
OR "data sanitization" OR anonymization OR pseudonymization
```

```
OR "privacy-preserving")
AND
("sensitive data" OR privacy OR security)
```

String C: Arquitetura intermediária e prevenção
Objetivo: Identificar arquiteturas e mecanismos relacionados.

```
("reverse proxy" OR "security proxy" OR "API gateway"
OR middleware OR "policy enforcement point")
AND
("data loss prevention" OR "data leakage prevention"
OR "content inspection" OR "data redaction")
AND
("sensitive data" OR "confidential information"
OR "personal data")
```

String D: Detecção de informações sensíveis
Objetivo: Identificar métodos de reconhecimento de credenciais, segredos, dados pessoais e outras informações sensíveis em código ou ambientes de desenvolvimento.

```
("sensitive data detection" OR "sensitive information detection"
OR "secret detection" OR "credential detection"
OR "PII detection" OR "API key detection")
AND
("source code" OR prompt OR IDE
OR "software development" OR "software repository")
```

String E: Impacto operacional
Objetivo: Encontrar métricas e evidências sobre impacto na experiência, produtividade, latência e utilidade dos assistentes.

```
("large language model" OR LLM)
AND
("code assistant" OR "AI-assisted programming")
AND
(privacy OR security)
AND
(usability OR latency OR utility OR productivity)
```

4 Levantamento e Análise de Requisitos

Este capítulo documenta o levantamento e a análise dos requisitos do mecanismo proposto: um proxy de segurança, posicionado entre as ferramentas de desenvolvimento assistidas por LLM e os provedores externos de IA, capaz de identificar e redigir dados sensíveis no tráfego HTTP em trânsito. O objetivo é tornar explícito *para quem* a solução é construída, *o que* ela deve fazer (requisitos funcionais) e *com que qualidade* deve fazê-lo (requisitos não funcionais), de modo que as decisões de projeto (Capítulo 5), a implementação (Capítulo 6) e os testes (Capítulo 7) possam ser rastreados até uma necessidade declarada.

4.1 Abordagem de Levantamento

O levantamento combinou quatro fontes de informação, citadas a seguir em ordem decrescente de peso na formulação dos requisitos:

1. **Lacunas da literatura.** A revisão sistemática (Capítulo 2) evidenciou que nenhum dos trabalhos analisados combina, ao mesmo tempo, a interceptação de tráfego HTTP em tempo real, o agnosticismo em relação à ferramenta cliente e ao provedor de destino e a avaliação do impacto operacional da camada de proteção. Esses três eixos originam, respectivamente, os requisitos de interceptação, de compatibilidade e de desempenho.

2. **Natureza do domínio.** O tráfego de assistentes de código é, em sua maioria, HTTPS com corpo JSON, e o conteúdo enviado pode incluir credenciais, *tokens* e identificadores internos. Isso impõe a terminação TLS local (e, portanto, a gestão de uma autoridade certificadora própria) e a análise do corpo da requisição.

3. **Prototipação evolutiva.** Os incrementos descritos na Seção 6.3 funcionaram como protótipos: parte dos requisitos (por exemplo, o modo de simulação e o diagnóstico do ambiente) emergiu do uso do próprio mecanismo em ferramentas reais.

4. **Análise documental do artefato.** A documentação e o código do repositório foram revisados para garantir que cada requisito esteja associado ao componente que o realiza e ao seu estado de implementação.

Os requisitos são identificados por **RF** (funcionais) e **RNF** (não funcionais) e priorizados pela técnica MoSCoW, na qual *Deve* indica requisito indispensável ao mecanismo cumprir sua finalidade, *Deveria* indica requisito importante, mas contornável, e *Poderia* indica requisito desejável. A coluna *Situação* registra o estado de realização: *Implementado* (presente no código), *Parcial* (presente com limitação conhecida, descrita no Capítulo 7) ou *Planejado*.

4.2 Identificação dos Usuários

Foram identificados cinco perfis, entre usuários diretos, usuários indiretos e partes interessadas. Os perfis foram construídos a partir do contexto de uso descrito na fundamentação teórica e das funcionalidades expostas pelo mecanismo; sua validação com usuários é tratada na Seção 8.2.5.

Tabela 2 – Perfis de usuários e partes interessadas

Perfil	**Necessidades e motivações**	**Interação com a solução**	**Tipo**
Desenvolvedor de software	Usar assistentes de código sem expor segredos do projeto; não perder produtividade nem alterar sua ferramenta de IA.	Inicia o proxy, aponta a ferramenta para a porta local, acompanha eventos na TUI e executa o diagnóstico do ambiente.	Usuário primário
Responsável por segurança e governança	Definir quais categorias de dados não podem sair da organização; comprovar, por registros, o que foi interceptado.	Edita padrões de detecção (embutidos e personalizados), configura modo e *hosts*, consulta a trilha de auditoria.	Usuário secundário
Mantenedor / contribuidor	Evoluir o código, publicar versões e manter a documentação consistente.	Compila, executa testes e *benchmarks*, publica binários e atualiza a documentação.	Usuário técnico
Provedor de LLM	Receber requisições válidas e íntegras, sem ser alterado.	Recebe a requisição já sanitizada; não interage diretamente com a solução.	Sistema externo
Avaliadores acadêmicos (orientação e banca)	Verificar reprodutibilidade, rastreabilidade e evidências do experimento.	Consultam documentação, repositório e registros de teste.	Parte interessada

Fonte: Elaborado pela equipe(2026).

4.2.1 Cenário de uso de referência

O cenário que orienta o projeto é o de um desenvolvedor que utiliza uma ferramenta de linha de comando ou uma extensão de IDE para pedir ajuda a um LLM sobre um trecho de código. Para fornecer contexto, a ferramenta monta uma requisição HTTPS

contendo o *prompt*, arquivos abertos e, eventualmente, o conteúdo de arquivos de configuração (como arquivos `.env`) que carregam chaves de API. O mecanismo proposto é executado na máquina do desenvolvedor; a ferramenta de IA é configurada para usá-lo como proxy; e, antes de a requisição alcançar o provedor, as credenciais reconhecidas são substituídas por um marcador, enquanto um registro de auditoria sem o conteúdo da requisição é gravado.

4.3 Requisitos Funcionais

O Tabela 3 lista os requisitos funcionais, a prioridade, o componente responsável (cf. Capítulo 5) e a situação atual.

Tabela 3 – Requisitos funcionais

ID	Descrição (o sistema deve…)	Prior.	Componente	Situação
RF01	atuar como proxy HTTP/HTTPS local, escutando em porta configurável (padrão 9000), para que ferramentas de IA possam ser apontadas para ele.	Deve	Proxy MITM	Implementado
RF02	gerar, na primeira execução, uma autoridade certificadora (CA) local e persistir o par certificado/chave, reutilizando-o nas execuções seguintes.	Deve	Gestor de CA	Implementado
RF03	instalar a CA no repositório de certificados do sistema operacional (macOS, Linux e Windows), informando falhas como aviso.	Deve	Instalador de CA	Implementado
RF04	terminar o TLS das conexões interceptadas, emitindo certificados assinados pela CA local, de modo a acessar o corpo das requisições.	Deve	Proxy MITM	Implementado
RF05	analisar apenas requisições `POST` ou `PUT` com corpo JSON (ou formulário), limitando a análise a corpos de até 5 MiB.	Deve	Filtro	Implementado
RF06	oferecer dois modos de operação: *whitelist* (somente *hosts* permitidos) e *passive* (todos os *hosts*, exceto os ignorados).	Deve	Filtro; Configuração	Implementado
RF07	ignorar *hosts* de telemetria e análise de uso configuráveis, com suporte a curingas.	Deve	Filtro	Implementado

ID	Descrição (o sistema deve…)	Prior.	Componente	Situação
RF08	identificar requisições destinadas a LLM pela inspeção do corpo (campos como `model`, `messages`, `contents`, `prompt`, `input`), e não pelo *host* de destino.	Deve	Filtro	Implementado
RF09	inferir o provedor e o modelo a partir do corpo e, na falta de indícios, a partir do *host*.	Deveria	Inferência de provedor	Implementado
RF10	detectar credenciais por padrões embutidos (chaves OpenAI, OpenAI de projeto, Anthropic, Google, GitHub, Slack, Discord e AWS).	Deve	Analisador	Implementado
RF11	permitir ativar ou desativar cada padrão embutido por configuração.	Deveria	Configuração	Implementado
RF12	aceitar padrões personalizados por expressão regular ou por lista de valores literais, cada um com rótulo e indicador de habilitação, ignorando com aviso os padrões inválidos.	Deve	Analisador; Configuração	Implementado
RF13	substituir cada ocorrência detectada pelo rótulo do padrão antes do encaminhamento, mantendo consistente o tamanho declarado do corpo.	Deve	Analisador; Proxy MITM	Implementado
RF14	oferecer modo de simulação (*dry-run*) que detecta e registra sem alterar a requisição.	Deveria	Analisador	Implementado
RF15	registrar trilha de auditoria em JSON Lines, com identificação de requisição, sessão e máquina, destino, provedor, modelo, padrões detectados, contagens e latências, sem gravar o conteúdo da requisição.	Deve	Auditoria	Implementado
RF16	registrar também as requisições ignoradas e os erros de conexão, devolvendo resposta de erro explícita ao cliente em caso de falha de conexão.	Deveria	Proxy MITM; Auditoria	Implementado
RF17	carregar configuração YAML opcional e, na sua ausência, operar com os padrões embutidos e a porta 9000.	Deve	Configuração	Implementado
RF18	oferecer interface de terminal (TUI) com painel em tempo real: contadores de requisições, redigidas e erros, provedores observados e últimos eventos.	Deveria	TUI	Implementado

ID	Descrição (o sistema deve…)	Prior.	Componente	Situação
RF19	permitir, na TUI, gerenciar padrões embutidos e personalizados (criar, editar, ativar, excluir), validando expressões regulares e persistindo a configuração.	Deveria	TUI; Configuração	Implementado
RF20	permitir, na TUI, editar porta, modo e listas de *hosts* do proxy, com confirmação de alterações não salvas.	Deveria	TUI; Configuração	Implementado
RF21	oferecer subcomando de diagnóstico que verifique a instalação da CA e a disponibilidade da porta.	Poderia	Diagnóstico	Implementado
RF22	oferecer subcomandos de ajuda e de versão.	Poderia	Ponto de entrada	Implementado
RF23	encerrar de forma controlada ao receber sinal de término, persistindo o registro de auditoria.	Deve	Ponto de entrada; Auditoria	Implementado
RF24	ser distribuído como binário para as plataformas suportadas, acompanhado de *checksums* SHA-256, com instalação assistida (Homebrew e *script* PowerShell).	Deveria	Distribuição (CI)	Implementado
RF25	permitir a extensão por *plugins* de padrões personalizados.	Poderia	—	Planejado

Fonte: Elaborado pela equipe (2026), a partir da análise do repositório do projeto.

4.4 Requisitos Não Funcionais

Os requisitos não funcionais foram formulados, sempre que possível, com um critério mensurável e um método de verificação, de forma a permitir a avaliação objetiva no Capítulo 7. A atenção especial a desempenho e a falsos positivos decorre da literatura revisada: o custo de latência e a taxa de falsos positivos determinam a viabilidade de adoção de qualquer camada de proteção no fluxo diário do desenvolvedor (TSEBAK; BANAKH; PISKOZUB, 2025).

Tabela 4 – Requisitos não funcionais

ID	Categoria	Requisito e critério de aceitação	Verificação
RNF01	Desempenho	A análise de uma requisição deve adicionar latência desprezível frente ao tempo de resposta de um LLM: média inferior a 3 ms por operação para corpos de até 1 MiB (o limiar de 3 ms é o do teste automatizado), medida com cargas de tamanho representativo.	Teste de latência; *benchmark* com cargas de 1 KB a 5 MiB
RNF02	Desempenho	O proxy deve processar corpos de até 5 MiB sem corromper a requisição encaminhada.	Teste de carga e de limite (planejado)
RNF03	Precisão	Nenhum falso positivo nos casos de dados benignos do conjunto de teste (tolerância zero) e detecção de todos os formatos de credencial suportados.	Testes de detecção
RNF04	Segurança	A chave privada da CA deve ser gerada localmente (RSA de 4096 bits), gravada com permissão restrita e protegida contra versionamento acidental.	Inspeção de código e do `.gitignore`
RNF05	Privacidade	O registro de auditoria não deve conter o conteúdo das requisições nem os segredos detectados; o proxy não deve transmitir dados a destinos diferentes dos requisitados pelo cliente.	Inspeção de código e do esquema do registro
RNF06	Portabilidade	Operar em macOS (Intel e Apple Silicon), Linux (amd64) e Windows (amd64).	Matriz de compilação do CI; testes manuais por plataforma
RNF07	Instalabilidade	Ser entregue como binário único, sem dependências de execução, instalável por gerenciador de pacotes ou *script*.	Inspeção; teste de instalação
RNF08	Usabilidade	A TUI deve funcionar em terminais de, no mínimo, 80 colunas, com atalhos documentados e validação imediata de entradas inválidas.	Teste com usuários

ID	Categoria	Requisito e critério de aceitação	Verificação
RNF09	Confiabilidade	Falhas de registro ou da TUI não devem bloquear o encaminhamento de requisições; conexões devem respeitar limites de tempo e falhas devem resultar em erro explícito.	Inspeção; teste manual de erro e de timeout
RNF10	Auditabilidade	Cada requisição processada deve gerar registro com identificador único de requisição, de sessão e de máquina, e carimbo de tempo em formato padronizado.	Inspeção do registro gerado
RNF11	Compatibilidade	Funcionar com qualquer ferramenta que permita configurar proxy, sem modificá-la, identificando requisições de IA pelo conteúdo.	Testes de identificação; teste de integração com ferramentas
RNF12	Configurabilidade	Padrões e modos de operação devem ser alteráveis sem recompilação.	Testes de configuração
RNF13	Manutenibilidade	Código organizado em pacotes coesos, com testes automatizados, integração contínua e documentação atualizada.	Inspeção; cobertura de testes
RNF14	Licenciamento	O código deve ser distribuído sob licença de código aberto (Apache 2.0).	Inspeção

Fonte: Elaborado pela equipe (2026).

4.5 Restrições, Premissas e Limitações Conhecidas

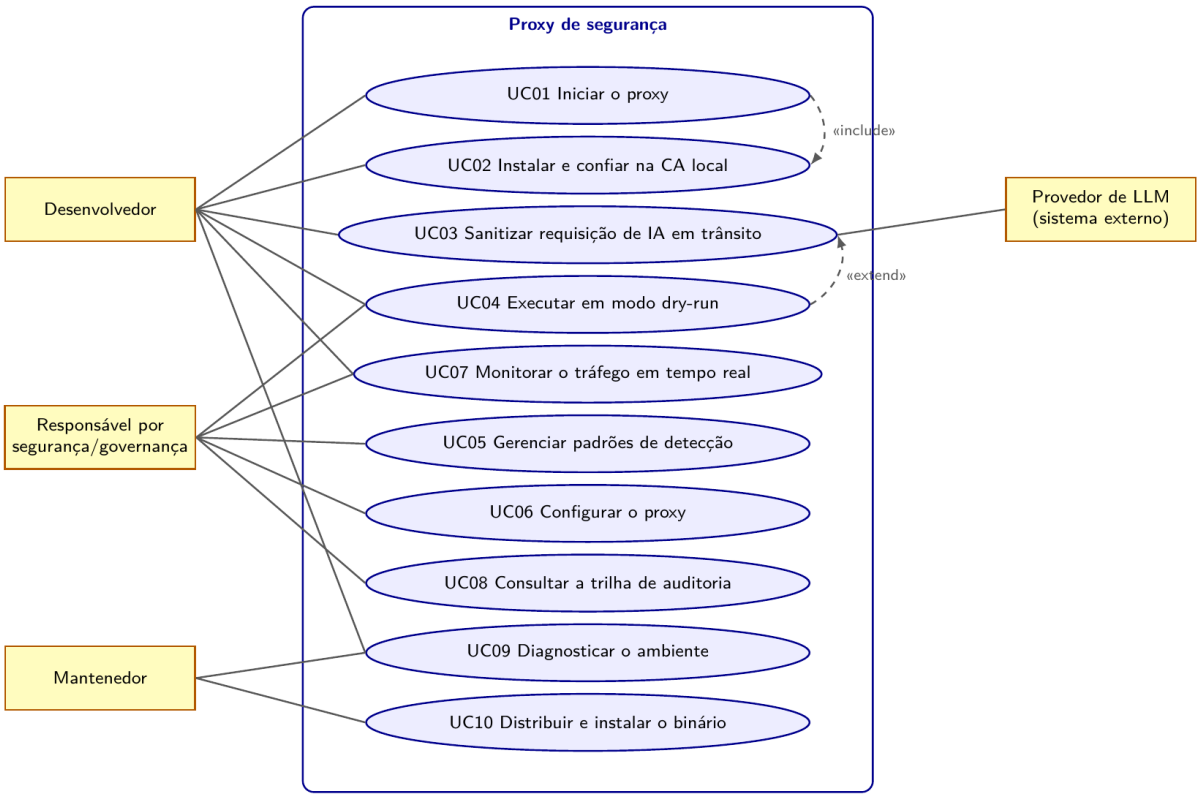
- **Confiança na CA local.** A interceptação de HTTPS só é possível porque o usuário instala e confia na CA gerada localmente, o que exige privilégios administrativos. A exposição da chave privada dessa CA permitiria a emissão de certificados válidos para qualquer domínio na máquina; por isso ela nunca deve sair do ambiente local (RNF04).
- **Ferramentas compatíveis.** Apenas ferramentas que aceitam proxy HTTP(S) configurável e que confiam na CA do sistema (ou em uma CA adicional) podem ser protegidas. Clientes que fixam certificados (*certificate pinning*) rejeitariam a interceptação.

- **Escopo da inspeção.** Somente o corpo das requisições de saída é inspecionado e redigido; respostas dos provedores trafegam sem análise. A detecção baseia-se em padrões (expressões regulares e literais), não em classificação semântica.
- **Ambiente.** A implementação utiliza a linguagem Go; o proxy opera na máquina do desenvolvedor, sem componente de servidor centralizado.

4.6 Casos de Uso

A Figura 1 apresenta os atores e os casos de uso principais, e o Tabela 5 resume cada caso de uso e os requisitos que ele realiza.

Figura 1 – Diagrama de casos de uso



Fonte: Elaborado pela equipe (2026).

4.7 Análise e Priorização

Dos 25 requisitos funcionais, 14 foram classificados como *Deve*, 8 como *Deveria* e 3 como *Poderia*. Os requisitos *Deve* formam o núcleo da proposta e concentram-se nos três eixos da lacuna identificada na literatura: interceptação em tempo real (RF01–RF05), identificação independente de ferramenta e de provedor (RF06–RF08)

Tabela 5 – Resumo dos casos de uso

ID	Caso de uso	Fluxo principal	Requisitos
UC01	Iniciar o proxy	O desenvolvedor executa o binário (com ou sem TUI); o sistema garante a CA, carrega a configuração e passa a escutar na porta definida.	RF01, RF02, RF17, RF18, RF23
UC02	Instalar e confiar na CA local	Na primeira execução, o sistema gera a CA e tenta instalá-la no repositório do sistema operacional, exigindo privilégios.	RF02, RF03
UC03	Sanitizar requisição de IA em trânsito	A ferramenta envia a requisição; o sistema a identifica como requisição de IA, detecta dados sensíveis, redige as ocorrências e encaminha ao provedor.	RF04–RF10, RF13, RF16
UC04	Executar em modo *dry-run*	Variante do UC03 em que as ocorrências são apenas detectadas e registradas, sem alteração do corpo.	RF14
UC05	Gerenciar padrões de detecção	O responsável por segurança ativa, desativa, cria, edita ou exclui padrões na TUI ou no arquivo de configuração.	RF11, RF12, RF19
UC06	Configurar o proxy	O responsável define porta, modo e listas de *hosts*.	RF06, RF07, RF17, RF20
UC07	Monitorar o tráfego em tempo real	O usuário acompanha, na TUI, contadores e últimos eventos.	RF18
UC08	Consultar a trilha de auditoria	O responsável analisa o arquivo de auditoria para verificar o que foi detectado e redigido.	RF15, RF16
UC09	Diagnosticar o ambiente	O usuário executa o diagnóstico para verificar CA e porta.	RF21, RF22
UC10	Distribuir e instalar o binário	O mantenedor publica versões e o usuário instala por pacote ou binário.	RF24

Fonte: Elaborado pela equipe (2026).

e detecção com redação seletiva (RF10–RF13). A trilha de auditoria (RF15) e o encerramento controlado (RF23) completam o conjunto indispensável, por sustentarem a avaliação do impacto operacional e a governança.

A seletividade da proteção, isto é, a possibilidade de calibrar *o que* é redigido, foi tratada como requisito de primeira ordem (RF11 e RF12). Essa decisão ecoa a observação de que aplicar uma transformação uniforme a todo o conteúdo comprometeria a utilidade do assistente (PAN; WU, 2025), e é refletida no critério de tolerância zero a falsos positivos (RNF03).

Os requisitos de plataforma (RF18 a RF24) foram priorizados abaixo do núcleo porque não alteram a capacidade de proteção, mas determinam a adoção prática; por isso, a avaliação com usuários (Seção 8.2.5) os contempla. O único requisito não implementado, RF25, depende da estabilização das interfaces internas de padrões e foi deliberadamente adiado.

5 Projeto da Solução

Este capítulo apresenta o projeto do mecanismo: a arquitetura, a modelagem dos dados persistidos e os diagramas que descrevem estrutura e comportamento. As decisões aqui descritas atendem aos requisitos do Capítulo 4; o registro cronológico e a justificativa de cada decisão estão na Seção 6.5.

5.1 Princípios de Projeto

A lacuna identificada na literatura (Capítulo 2) orienta cinco princípios:

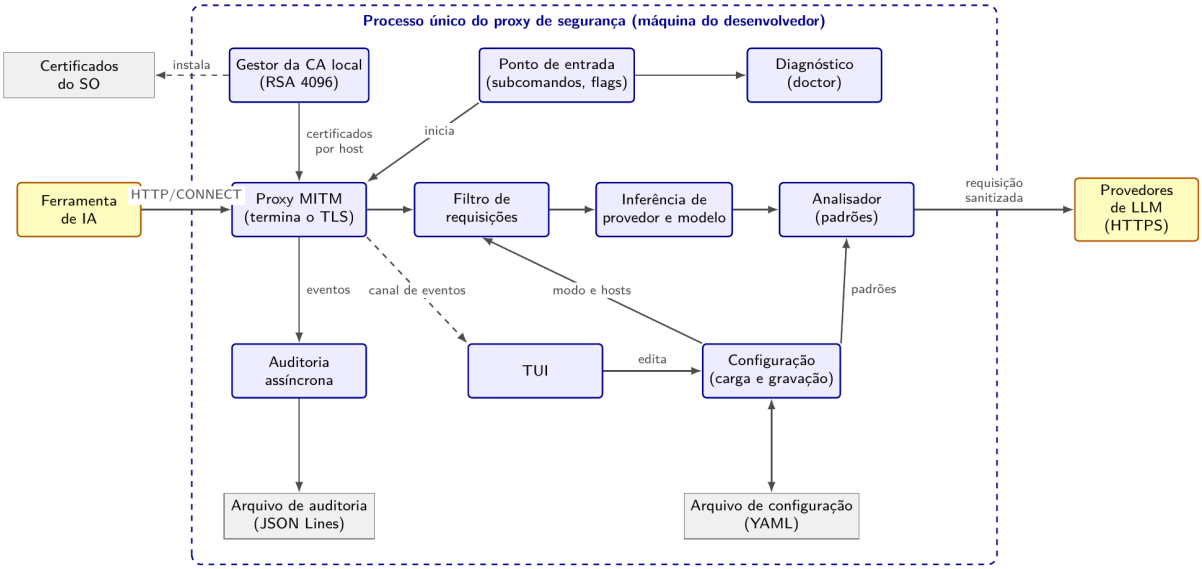
- **Intermediação transparente:** a proteção é aplicada em um ponto de aplicação de política situado entre o cliente e o provedor, sem alterar a ferramenta de desenvolvimento, em linha com o padrão de proxies reversos e *gateways* descrito na Seção 2.9.
- **Agnosticismo:** requisições de IA são reconhecidas pelo conteúdo, e não pelo destino, para funcionar com ferramentas e provedores distintos (RF08, RNF11).
- **Seletividade:** padrões embutidos e personalizados podem ser ativados individualmente, evitando redação indiscriminada (RF11, RF12).
- **Execução local e mínima exposição:** o processamento ocorre na máquina do desenvolvedor e o registro de auditoria não contém o conteúdo das requisições (RNF04, RNF05).
- **Não interferência:** registro e interface são desacoplados do caminho crítico, de modo que sua lentidão não atrase as requisições (RNF09).

5.2 Arquitetura

5.2.1 Visão geral

A solução é um único processo, compilado em um binário, que combina um proxy HTTP/HTTPS com terminação TLS e um *pipeline* de filtragem e análise. A Figura 2 mostra os componentes e o fluxo de dados, e a Tabela 6 descreve a responsabilidade de cada um e os requisitos que realiza. A organização em pacotes do código-fonte é apresentada na Seção 6.2.

Figura 2 – Arquitetura do mecanismo proposto



Fonte: Elaborado pela equipe (2026). Amarelo: entidades externas; azul: componentes; cinza: armazenamento local.

5.2.2 Estilo arquitetural

O estilo é o de um *proxy* intermediário com *pipeline* de filtros: cada requisição atravessa estágios sucessivos (filtragem, inferência, análise e registro), e cada estágio tem uma única responsabilidade. Os componentes de configuração, auditoria e interface são periféricos ao caminho crítico e se comunicam com o núcleo por estruturas explícitas: a configuração é lida na inicialização, os eventos são enviados à auditoria por uma fila com capacidade fixa e à TUI por um canal com capacidade fixa. Se a fila de auditoria ou o canal da TUI estiverem cheios, o evento é descartado em vez de bloquear a requisição; essa escolha privilegia a latência e a disponibilidade do fluxo do desenvolvedor sobre a completude do registro e é discutida como decisão arquitetural na Seção 6.5.

5.2.3 Fluxo de processamento de uma requisição

A Figura 3 apresenta a sequência de interações para uma requisição HTTPS de uma ferramenta de IA. O cliente solicita um túnel (`CONNECT`); o proxy responde apresentando um certificado assinado pela CA local e passa a receber a requisição decifrada. Em seguida, o proxy consulta o filtro e a inferência de provedor, aciona o analisador, publica o evento para a auditoria sem esperar sua conclusão e encaminha ao provedor a requisição já sanitizada. A resposta retorna ao cliente sem inspeção.

As Figuras 4 e 5 detalham a lógica de decisão. A primeira trata da pré-filtragem: apenas requisições `POST` ou `PUT` com corpo JSON ou de formulário, destinadas a *hosts* não ignorados e permitidos pelo modo de operação, cujo corpo contenha campos ca-

Tabela 6 – Componentes da arquitetura

Componente	**Responsabilidade**	**Requisitos**
Ponto de entrada	Interpreta subcomandos e *flags* (execução, TUI, *dry-run*, diagnóstico, versão), orquestra a inicialização e o encerramento controlado.	RF01, RF22, RF23
Gestor de CA	Gera e persiste o par certificado/chave da CA local; instala o certificado no repositório do sistema, com implementação específica por sistema operacional.	RF02, RF03
Proxy MITM	Escuta na porta configurada, termina o TLS com certificados emitidos pela CA local, aplica o *pipeline* a cada requisição e encaminha o resultado ao provedor; trata erros de conexão e limites de tempo.	RF01, RF04, RF13, RF16
Filtro de requisições	Decide se uma requisição deve ser analisada (método, tipo de conteúdo, listas de *hosts*, modo de operação, presença de campos de IA no corpo).	RF05–RF08
Inferência de provedor	Deduz provedor e modelo a partir do corpo e, como alternativa, do *host*.	RF09
Analisador	Aplica os padrões compilados ao corpo; substitui ocorrências pelo rótulo ou apenas as contabiliza, no modo *dry-run*.	RF10, RF12–RF14
Configuração	Carrega o arquivo YAML (ou os padrões por omissão), compila padrões e persiste alterações feitas pela TUI.	RF11, RF12, RF17
Auditoria	Converte eventos em registros JSON e os grava de forma assíncrona, por um conjunto de *workers*.	RF15, RF16
TUI	Exibe o painel em tempo real e telas de edição de padrões e do proxy.	RF18–RF20
Diagnóstico	Verifica a instalação da CA e a disponibilidade da porta.	RF21

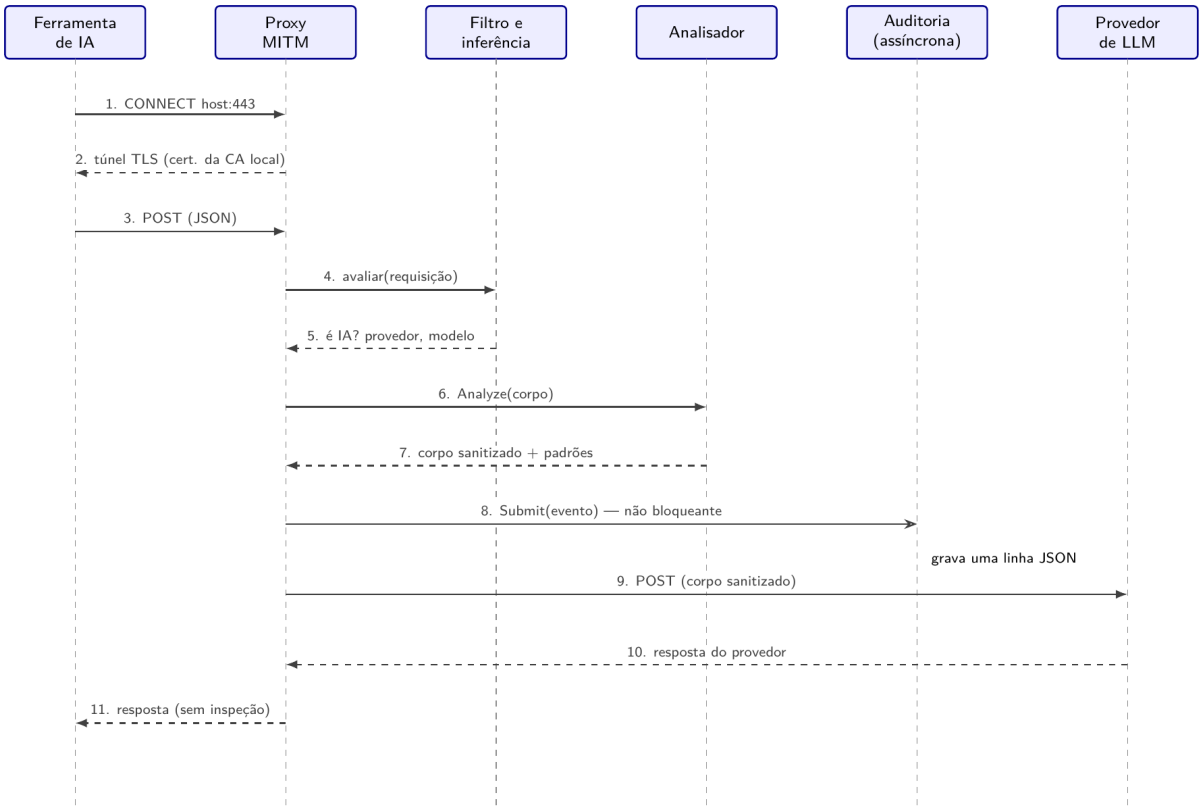
Fonte: Elaborado pela equipe (2026).

racterísticos de requisições de IA, avançam para a análise; as demais são registradas como ignoradas e encaminhadas sem alteração. A segunda trata da análise e da sanitização: se algum padrão for detectado, o corpo é substituído (ou mantido, no modo *dry-run*) e o evento é registrado antes do encaminhamento.

5.2.4 Concorrência e limites operacionais

O servidor atende conexões de forma concorrente, e o registro de auditoria utiliza um *pool* de quatro *workers* sobre uma fila com capacidade para cem eventos; o canal de eventos da TUI comporta 128 itens. O servidor define limites de 30 s para leitura e escrita e de 60 s para conexões ociosas, e falhas de conexão em túneis resultam

Figura 3 – Diagrama de sequência do processamento de uma requisição de IA



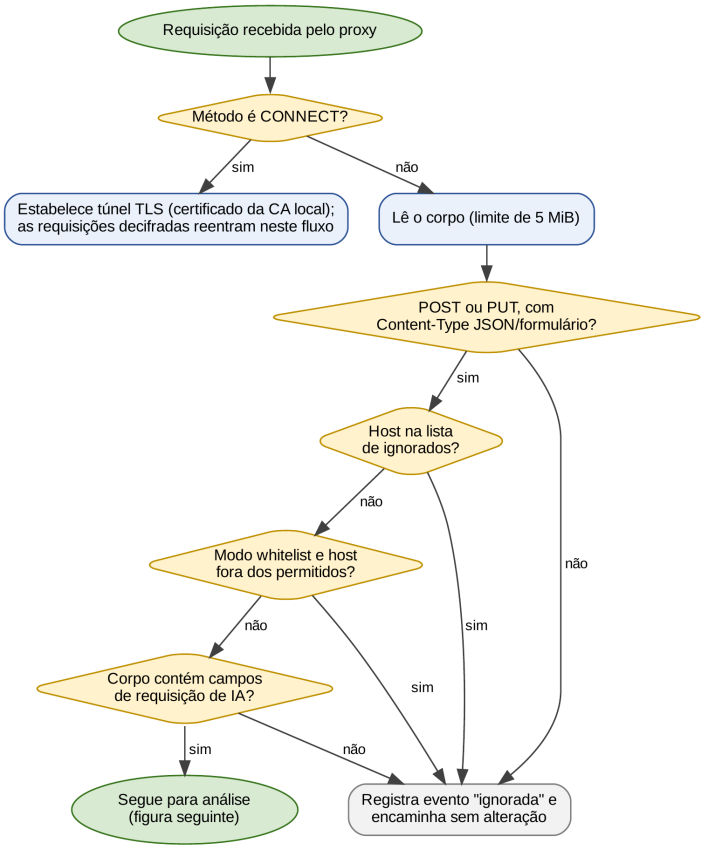
Fonte: Elaborado pelo autor (2026). Setas tracejadas: retornos; seta de ponta aberta: chamada assíncrona.

em resposta `502` ao cliente e em um registro de erro; em requisições HTTP simples, a verificação registrou resposta de erro com status 500 (Seção 7.3). A análise é limitada a corpos de até 5 MiB. Nas conexões interceptadas, a negociação é restrita ao HTTP/1.1.

5.2.5 Implantação e segurança

O mecanismo é implantado na própria máquina do desenvolvedor, e a ferramenta de IA é configurada para usá-lo como proxy (por exemplo, por meio das variáveis de ambiente de proxy ou de configuração da ferramenta). A CA local é o ativo mais sensível: a chave privada é gerada com 4096 bits, gravada com permissão restrita ao proprietário e excluída do controle de versão, e o certificado possui validade de dez anos. Os riscos principais, já mapeados na documentação do projeto, são o vazamento da chave da CA e o comprometimento do binário; as mitigações são a manutenção da chave exclusivamente local, a verificação de integridade dos binários por *checksum* SHA-256 e a distribuição apenas por fontes oficiais. A seção de problemas (Seção 7.4) registra ainda um *script* de conveniência que desativa a verificação de certificados no processo da ferramenta cliente, prática que deve ser substituída pela confiança explícita na CA.

Figura 4 – Fluxograma da pré-filtragem de requisições



Fonte: Elaborado pela equipe (2026).

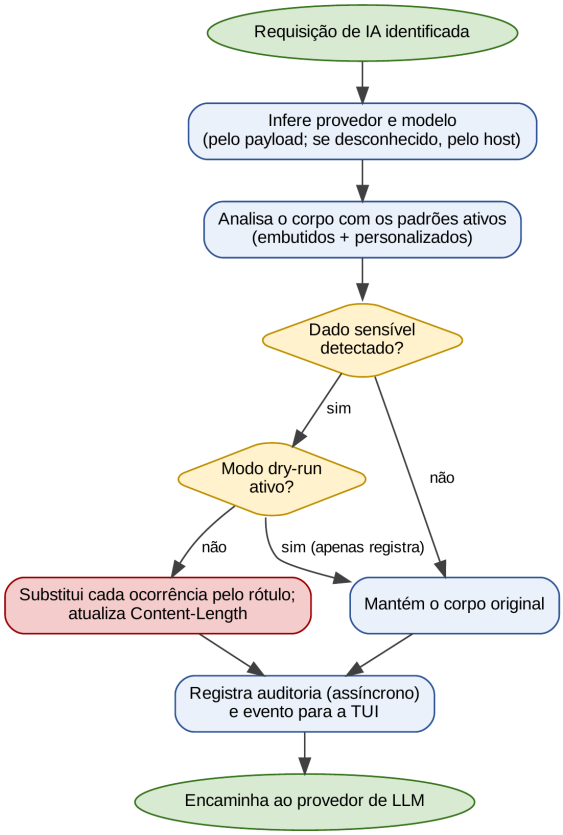
5.3 Modelagem dos Dados

5.3.1 Decisão sobre o armazenamento

O mecanismo não utiliza um sistema gerenciador de banco de dados (SGBD). A decisão decorre do perfil dos dados: (i) a configuração é pequena, editada por humanos e lida uma única vez na inicialização; (ii) o registro de auditoria é uma sequência de eventos imutáveis, gravados apenas por inclusão e lidos por ferramentas externas; (iii) o requisito de instalação como binário único, sem dependências e sem serviços em segundo plano (RNF07), tornaria a inclusão de um SGBD um custo de implantação desproporcional ao ganho. Em contrapartida, não há consultas ad hoc nativas, integridade referencial imposta pelo armazenamento nem controle de concorrência entre processos; as consequências dessa escolha são tratadas na Seção 5.3.4.

Assim, a persistência é feita por três artefatos: um arquivo YAML de configuração, um arquivo de auditoria no formato JSON Lines (um objeto JSON por linha) e o par de arquivos PEM da CA. Para fins de projeto, os dados foram modelados com a notação

Figura 5 – Fluxograma da análise e da sanitização



Fonte: Elaborado pelo autor (2026).

entidade-relacionamento, de modo que o modelo seja independente do formato físico e possa ser mapeado para um banco relacional, caso a evolução do projeto o justifique.

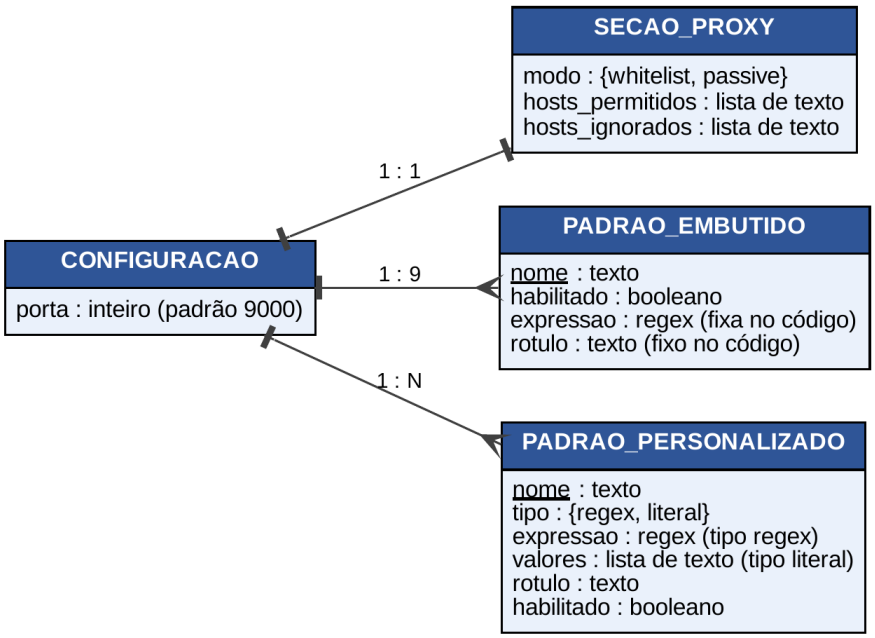
5.3.2 Modelo de configuração

A Figura 6 mostra o modelo da configuração. A configuração possui exatamente uma seção de proxy, nove definições de padrões embutidos (cuja habilitação é controlada por oito indicadores, pois os padrões de chave de acesso e de chave secreta da AWS compartilham um único indicador) e um número arbitrário de padrões personalizados. As expressões e os rótulos dos padrões embutidos são constantes do código, e somente o indicador de habilitação é configurável.

5.3.3 Modelo de auditoria

A Figura 7 mostra o modelo do registro de auditoria. Cada execução do proxy define uma sessão, identificada por um código de oito caracteres hexadecimais derivado do

Figura 6 – Modelo entidade-relacionamento da configuração



Fonte: Elaborado pela equipe (2026).

instante de início, e uma máquina, identificada por um *hash* truncado do nome do computador. Cada requisição processada, ignorada ou com erro gera uma entrada. Na implementação, a lista de padrões detectados é armazenada como *vetor* dentro da própria entrada (forma desnormalizada, natural em JSON); no modelo conceitual, ela é representada como a entidade associativa `PADRAO_DETECTADO`, o que explicita a relação de um para muitos.

Tabela 8 – Dicionário de dados da entrada de auditoria (campos JSON)

Campo	**Tipo**	**Descrição**
`timestamp`	texto	Instante do evento em RFC 3339.
`request_id`	texto	Identificador único da requisição.
`session_id`, `machine_id`	texto	Identificadores da sessão e da máquina.
`host`, `path`, `method`	texto	Destino e método da requisição.
`provider`, `model`	texto	Provedor inferido (ou `skipped` para requisições ignoradas) e modelo informado.
`redacted`	booleano	Indica se houve ocorrência detectada.
`pattern_count`	inteiro	Total de ocorrências detectadas.

Campo	Tipo	Descrição
`detected_patterns`	lista de texto	Nomes dos padrões que dispararam.
`redaction_positions`	lista de texto	Localização das ocorrências (atualmente o valor genérico `body`).
`message_count`	inteiro	Número de mensagens do corpo.
`has_system_prompt`, `stream`	booleano	Presença de instrução de sistema e uso de resposta em fluxo.
`request_body_size`	inteiro	Tamanho do corpo da requisição.
`response_body_size`, `response_status`	inteiro	Previstos para a resposta; hoje não preenchidos.
`proxy_latency_ms`	inteiro	Tempo decorrido até a conclusão do processamento da requisição pelo proxy, em milissegundos.
`analysis_duration_ms`	inteiro	Duração da análise do corpo, em milissegundos.
`proxy_error`, `error_message`	booleano; texto	Indicam falha de conexão e sua descrição.
`was_blocked`	booleano	Previsto para bloqueio de requisição; hoje não utilizado.

Fonte: Elaborado pela equipe (2026), a partir da definição do registro no código-fonte.

Dois aspectos do modelo são relevantes para a integridade dos dados: o registro não contém o conteúdo da requisição nem os valores detectados, apenas metadados e contagens (RNF05); e, por ser um arquivo de inclusão, não há atualização ou exclusão de entradas pelo mecanismo.

5.3.4 Mapeamento relacional proposto

A ausência de um SGBD limita as análises que o responsável por governança poderia realizar sobre a trilha de auditoria (por exemplo, quais padrões são mais frequentes por provedor ao longo do tempo). Como proposta de evolução, não implementada nesta etapa, o modelo da Figura 7 pode ser mapeado para um banco relacional embarcado, que preservaria a instalação como binário único. O Listagem 5.1 apresenta o esquema sugerido, com a lista de padrões normalizada e índices por instante e por provedor.

Listing 5.1 – Esquema relacional proposto para a trilha de auditoria

```
CREATE TABLE sessao (
  id_sessao  TEXT PRIMARY KEY,
  id_maquina TEXT NOT NULL
);
```

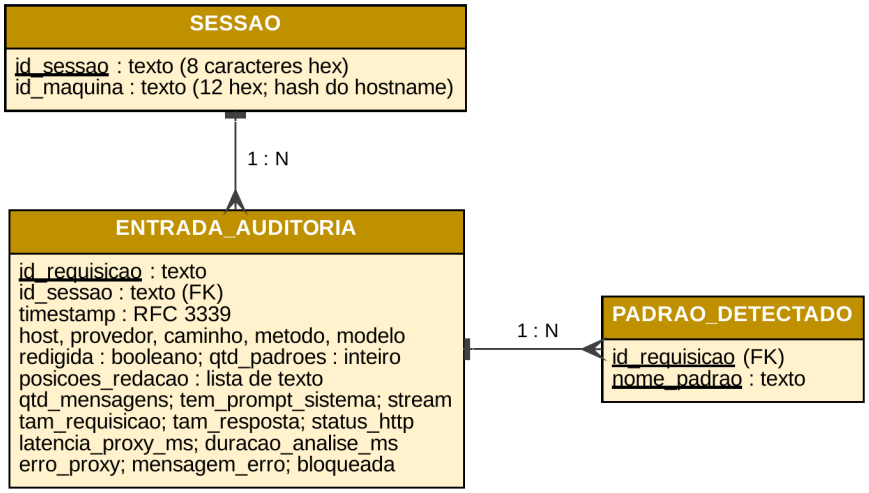
Tabela 7 – Dicionário de dados da configuração (arquivo YAML)

Campo (caminho YAML)	**Tipo**	**Descrição e regras**
`port`	inteiro	Porta de escuta; valor ausente ou zero equivale a 9000.
`proxy.mode`	texto	`whitelist` (padrão) ou `passive`.
`proxy.allowed_hosts`	lista de texto	*Hosts* analisados no modo *whitelist*; admite curinga `*.dominio`.
`proxy.skip_hosts`	lista de texto	*Hosts* sempre ignorados; admite curingas (por exemplo, `*.telemetry.*`).
`analyzer.built_in.`	booleano	Habilita ou desabilita cada padrão embutido (oito indicadores).
`analyzer.custom_patter`	texto	Nome do padrão personalizado.
`...type`	texto	`regex` ou `literal`; outros valores são ignorados com aviso.
`...pattern`	texto	Expressão regular (tipo `regex`); se inválida, o padrão é ignorado com aviso.
`...values`	lista de texto	Valores exatos (tipo `literal`); valores vazios são descartados.
`...label`	texto	Marcador que substitui cada ocorrência detectada.
`...enabled`	booleano	Indica se o padrão participa da análise.

Fonte: Elaborado pela equipe (2026).

```
CREATE TABLE entrada_auditoria (
  id_requisicao     TEXT PRIMARY KEY,
  id_sessao         TEXT NOT NULL REFERENCES sessao(id_sessao),
  ocorrido_em       TIMESTAMP NOT NULL,
  host              TEXT NOT NULL,
  provedor          TEXT NOT NULL,
  caminho           TEXT,
  metodo            TEXT NOT NULL,
  modelo            TEXT,
  redigida          BOOLEAN NOT NULL,
  qtd_padroes       INTEGER NOT NULL DEFAULT 0,
  qtd_mensagens     INTEGER,
  tem_prompt_sistema BOOLEAN,
  stream            BOOLEAN,
  tam_requisicao    INTEGER,
  status_http       INTEGER,
  latencia_proxy_us INTEGER,
  duracao_analise_us INTEGER,
  erro_proxy        BOOLEAN NOT NULL DEFAULT FALSE,
  mensagem_erro     TEXT
);
CREATE TABLE padrao_detectado (
  id_requisicao TEXT NOT NULL REFERENCES entrada_auditoria(id_requisicao),
  nome_padrao   TEXT NOT NULL,
  PRIMARY KEY (id_requisicao, nome_padrao)
);
CREATE INDEX ix_auditoria_tempo    ON entrada_auditoria(ocorrido_em);
CREATE INDEX ix_auditoria_provedor ON entrada_auditoria(provedor, redigida);
```

Figura 7 – Modelo entidade-relacionamento do registro de auditoria



Fonte: Elaborado pela equipe (2026).

O esquema substitui os campos de latência em milissegundos por campos em microssegundos, correção motivada pela limitação descrita na Seção 7.4 (análises com duração inferior a um milissegundo são registradas como zero). Os campos `was_blocked` e `redaction_positions` foram omitidos por não terem uso efetivo na versão atual.

5.4 Interfaces

5.4.1 Interface de linha de comando

Tabela 9 – Interface de linha de comando

Comando	**Efeito**
(sem argumentos)	Inicia o proxy em modo *headless*, com saída textual.
`--tui`	Inicia o proxy com a interface interativa de terminal.
`--dry-run`	Detecta e registra, sem modificar as requisições; combinável com `--tui`.
`doctor`	Executa o diagnóstico do ambiente (CA e porta).
`version`, `help`	Exibem a versão do binário e a ajuda.

Fonte: Elaborado pela equipe (2026).

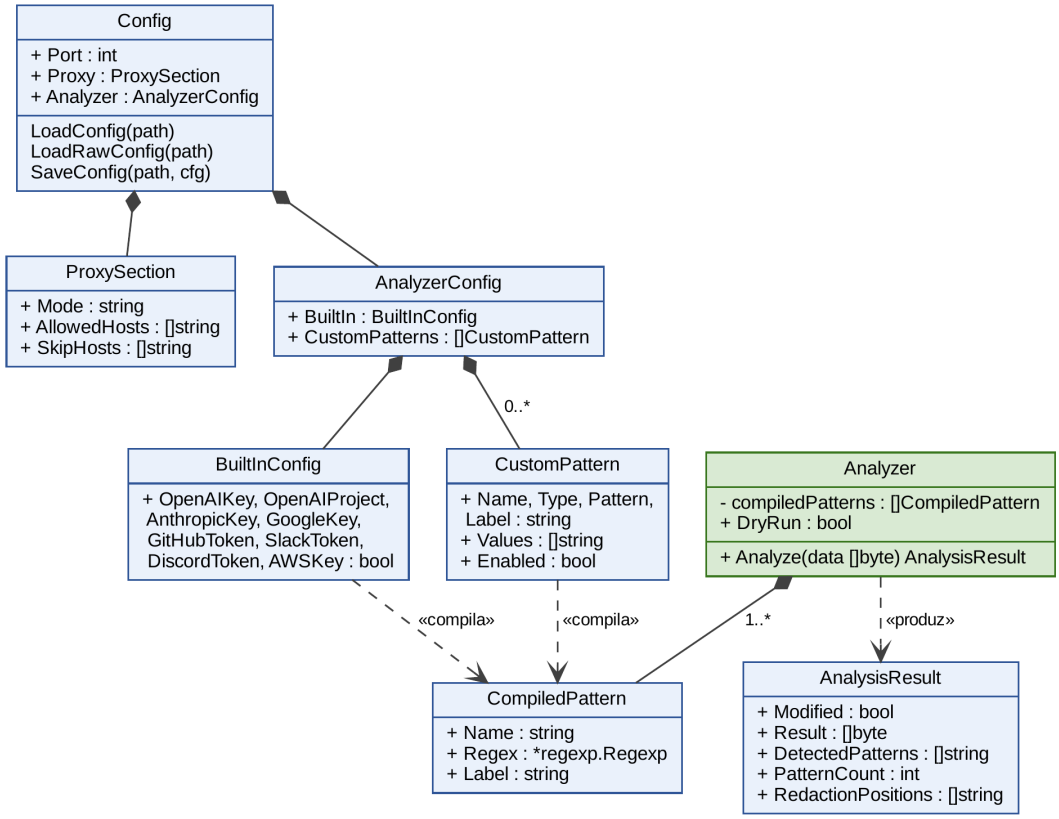
5.4.2 Interface de terminal (TUI)

A TUI é composta por três telas, navegáveis por atalhos de teclado: o *painel*, que exibe contadores (total, redigidas, erros), distribuição por provedor e os últimos eventos, além de indicador de modo *dry-run*; a tela de *padrões*, com a lista de padrões embutidos (alternância de habilitação) e personalizados (criação, edição e exclusão com validação imediata da expressão regular); e a tela de *configuração do proxy*, com porta, modo e listas de *hosts*. Alterações são gravadas no arquivo de configuração e passam a valer na próxima inicialização do proxy.

5.5 Diagrama de Classes

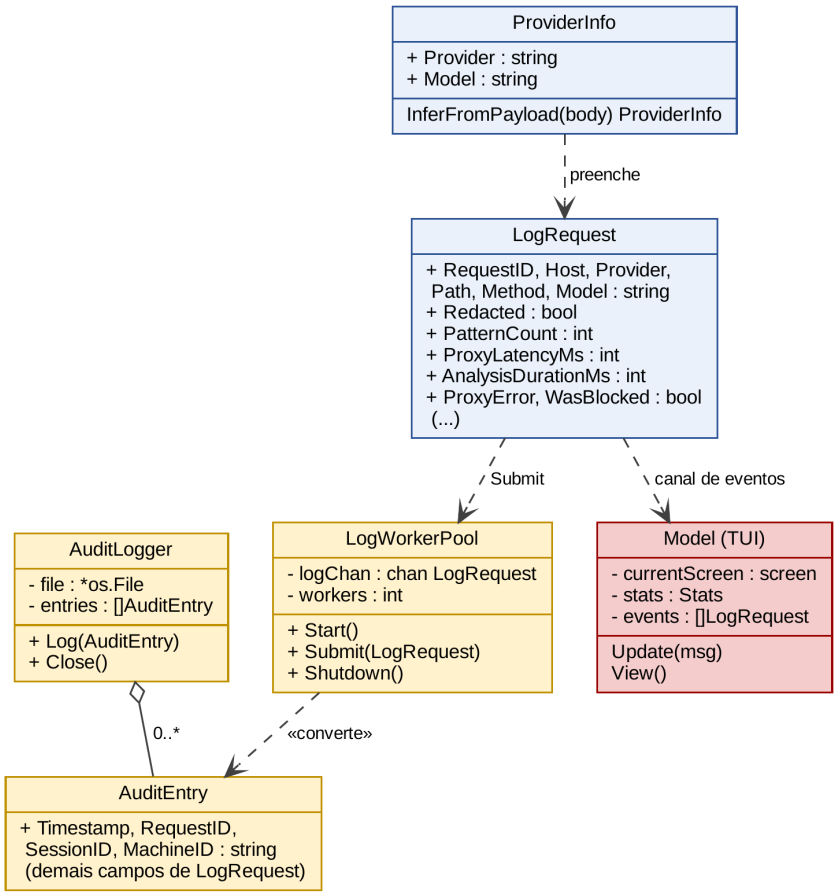
As Figuras 8 e 9 apresentam as principais estruturas do código, divididas em dois grupos: configuração e análise, e registro e interface. Os nomes foram mantidos como no código-fonte e os atributos, simplificados.

Figura 8 – Diagrama de classes: configuração e análise



Fonte: Elaborado pela equipe (2026).

Figura 9 – Diagrama de classes: registro e interface



Fonte: Elaborado pela equipe (2026).

O analisador é intencionalmente simples: mantém a lista de padrões compilados e um indicador de simulação, e sua operação principal recebe o corpo e devolve um resultado com o corpo transformado, os nomes dos padrões e a contagem de ocorrências. Essa interface estreita facilita a evolução prevista no requisito RF25, pois novos mecanismos de detecção podem ser incorporados sem alterar o proxy.

6 Desenvolvimento da Solução

Este capítulo descreve como a solução foi construída: o ambiente de desenvolvimento, a organização do repositório, a evolução por incrementos, o acompanhamento com a orientação, o registro das decisões e a manutenção da documentação. As informações sobre a evolução do código foram extraídas do histórico de versionamento, de modo que cada afirmação sobre o processo pode ser conferida por um identificador de *commit*.

6.1 Preparação do Ambiente

A Tabela 10 resume as tecnologias e ferramentas utilizadas. O ambiente foi mantido deliberadamente enxuto: o produto é um binário único, e a compilação depende apenas do compilador da linguagem e de duas bibliotecas diretas de domínio (proxy e interface de terminal), além de uma biblioteca de leitura de YAML.

Tabela 10 – Ambiente e ferramentas de desenvolvimento

Item	**Descrição**
Linguagem	Go (módulo declarado para a versão 1.25). Escolhida por gerar binário único multiplataforma, por oferecer biblioteca padrão para HTTP, TLS e certificados X.509.
Bibliotecas	Biblioteca de proxy HTTP com suporte a interceptação TLS (*goproxy*, v1.8.3); leitura e escrita de YAML (*yaml.v3*, v3.0.1); interface de terminal (*Bubble Tea* v1.3.10 e *Lip Gloss* v1.1.0).
Automação local	*Makefile* com alvos de compilação por plataforma, execução, diagnóstico, geração de *checksums*, desinstalação da CA, testes, cobertura e *benchmarks*.
Controle de versão	Git, com repositório público no GitHub.
Integração contínua	*Workflow* do GitHub Actions acionado por *tags* de versão, com matriz de compilação para macOS (Apple Silicon e Intel), Linux (amd64) e Windows (amd64), *CGO* desabilitado, geração de *checksums* SHA-256 e publicação da *release*.
Distribuição	Binários de *release*; instalação por Homebrew (macOS e Linux); *script* PowerShell (Windows).
Dados de teste	Arquivos com credenciais fictícias, no formato dos *tokens* reais mas sem validade, usados exclusivamente para testar a detecção.

Fonte: Elaborado pelo equipe (2026), a partir dos arquivos de configuração do repositório.

6.2 Organização do Repositório

O repositório adota uma estrutura convencional para projetos em Go, com o ponto de entrada em `cmd/` e o código de domínio em `internal/`, o que impede sua importação por outros módulos e reforça os limites entre pacotes.

Listing 6.1 – Estrutura do repositório (simplificada)

```
.
|-- cmd/<aplicacao>/main.go    # ponto de entrada: subcomandos, flags, orquestracao
|-- internal/
|   |-- ca/                    # geracao da CA local e instalacao no macOS
|   |-- guardian/              # nucleo: proxy MITM, filtro, provedor, analisador,
|   |                          #   configuracao, auditoria, instalacao de CA (Linux/
    Windows)
|   |-- doctor/                # diagnostico do ambiente
|   `-- tui/                   # interface de terminal (Bubble Tea)
|-- scripts/                   # scripts de conveniencia para iniciar ferramentas
    com proxy
|-- markdown/                  # documentacao complementar (esquema de auditoria,
    testes)
|-- .github/workflows/         # integracao continua e publicacao de releases
|-- Makefile, go.mod, go.sum
|-- config.yml.example         # exemplo comentado de configuracao
`-- readme.md, SECURITY.md, CONTRIBUTING.md, LICENSE
```

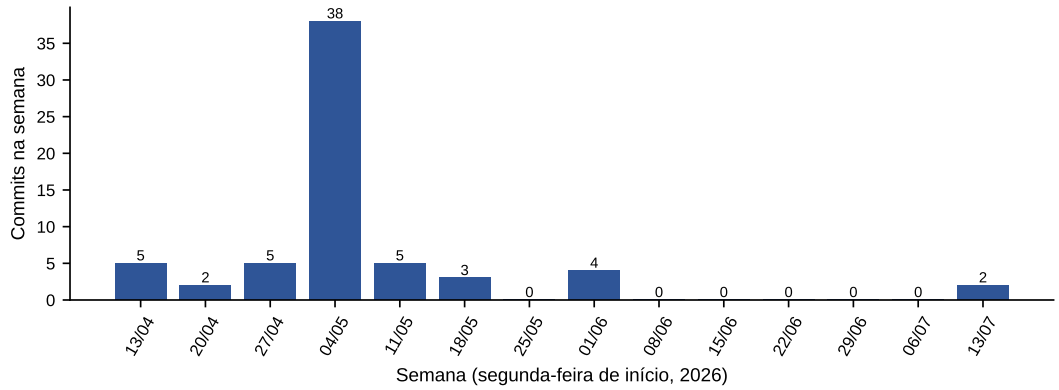
Convenções. As mensagens de *commit* seguem, na maior parte do histórico, prefixos que indicam o tipo de alteração (*feat*, *fix*, *doc*, *test*, *ci*, *refact*). O guia de contribuição estabelece dois ramos de longa duração, um estável e um de desenvolvimento, com *pull requests* de funcionalidades dirigidos a este último, além de exigir testes para novas funcionalidades e execução de *benchmarks* quando o analisador ou o núcleo forem alterados. O repositório contém ainda política de segurança (relato de vulnerabilidades) e licença Apache 2.0. Arquivos de certificado, binários, *checksums* e registros de auditoria são excluídos do versionamento.

Dimensão. O código-fonte contém cerca de 3.600 linhas de Go de produção e 1.300 linhas de testes, distribuídas em quatro pacotes internos.

6.3 Implementação por Incrementos

O desenvolvimento seguiu um processo iterativo e incremental, sem *sprints* formais. O histórico reúne 64 *commits*, de 16 de abril a 17 de julho de 2026. A Tabela 11 agrupa esses *commits* em nove incrementos, definidos por coesão de objetivo, e a Figura 10 mostra a distribuição semanal.

Figura 10 – Atividade de desenvolvimento por semana



Fonte: Elaborado pela equipe (2026), a partir do histórico de *commits*.

Tabela 11 – Incrementos de desenvolvimento

Inc.	**Período**	**Objetivo e entregas**	**Evidência (*commits*)**	**Requisitos**
I0	16–21/04 (7 *commits*)	**Exploração.** Protótipo de monitoramento TCP/HTTP, captura de tráfego em arquivo de pacotes com relatório em *script* Python e tentativa de configuração por regras de *firewall*. Primeira versão do analisador.	743885b, d47a2d8, 1236bda, 6cebe4b, 2d8a085, 4de235e	—
I1	29–30/04 (3)	**Primeira versão funcional.** Analisador por expressões regulares, registro de auditoria em arquivo, métricas e primeira documentação. Remoção do protótipo de captura de pacotes.	8523640, 913d304, 7a7a1f1	RF10, RF13, RF15
I2	01–03/05 (2)	**CA local e configuração.** Geração da CA e instalação no macOS; leitura de configuração YAML com padrões personalizados e primeiro filtro de requisições.	3683963, bdb96f0	RF02, RF03, RF12, RF17
I3	04/05 (9)	**Unificação e infraestrutura.** Consolidação do repositório; *Makefile*; instaladores de CA para Linux e Windows; licença; *workflow* de publicação; especificação do certificado.	3fc3d71, 9317162, 93ab52a, c23afe3, 3964fb7	RF03, RF24

Inc.	Período	Objetivo e entregas	Evidência (*commits*)	Requisitos
I4	05/05 (6)	**Testes de desempenho e de padrões.** *Benchmark*, testes de latência e de vazão, testes de padrões personalizados e ajustes do exemplo de configuração.	ef79c09, 5bd4a00, 2d9942e, 18832a7, 7b154ff	RNF01, RNF03
I5	06–08/05 (23)	**Integração, endurecimento e documentação.** Integração com uma segunda ferramenta de linha de comando; política de segurança e guia de contribuição; arquivo de proxy personalizado; instalação automática da CA no Linux; diagnóstico; comandos *make*; desinstalação da CA; modo *dry-run*; seção de riscos de segurança.	f629dd8, cd10441, 6f77577, d2c13a7, ea07ba6, 2be8767, 81de42e, aee374a, f3b3051	RF03, RF14, RF21, RF22
I6	12–18/05 (8)	**Refatoração do proxy.** Identificação de requisições de IA pelo corpo, inferência de provedor, modos *whitelist* e *passive*, limite de corpo e 370 linhas de testes de filtro; correções de CI; ajustes de documentação e demonstração.	6a234ce, bb9872f, 8ad5a6e, 98dfe25	RF05–RF09
I7	02–03/06 (4)	**Interface de linha de comando e TUI.** Subcomandos e *flags*; painel em tempo real; edição de padrões e do proxy com persistência no YAML; *script* de instalação para Windows; pré-lançamento.	0982721, 72e0282, 925bbc4, 0a1158e	RF18–RF20, RF24
I8	17/07 (2)	**Distribuição e limpeza.** Documentação da instalação por Homebrew e remoção de arquivos sem uso.	137478e, faebae9	RF24

Fonte: Elaborado pela equipe (2026), a partir do histórico de versionamento.

Três observações caracterizam o processo. Primeiro, o incremento I0 foi um *spike* exploratório cuja principal contribuição foi negativa: a captura passiva de pacotes não permite inspecionar o conteúdo de tráfego TLS, o que motivou a adoção do proxy com terminação TLS (decisão D01, Seção 6.5). Segundo, houve uma concentração de esforço em 4–8 de maio (38 dos 64 *commits*), com predominância de infraestrutura e documentação. Terceiro, os requisitos centrais (RF10 a RF13) foram entregues cedo (I1), e os incrementos seguintes ampliaram a abrangência (I6), a operacionalidade (I3, I5) e a usabilidade (I7), o que é coerente com a priorização MoSCoW do Capítulo 4.

6.4 Reuniões de Acompanhamento

O acompanhamento com a orientação é registrado em formato de ata resumida, com data, pauta, deliberações e encaminhamentos. A Tabela 12 reúne as orientações recebidas que já foram incorporadas ao trabalho e reserva linhas para as demais reuniões.

Tabela 12 – Registro de reuniões de acompanhamento e orientações incorporadas

Data	**Pauta**	**Deliberação / orientação**	**Encaminhamento**
27/08/2026	Critérios da revisão da literatura	Todas as referências devem possuir DOI ou ISSN; excluir TCCs e documentos sem revisão por pares.	Aplicado à seleção e ao arquivo de referências (Capítulo 2).
27/08/2026	Justificativa do problema	Fundamentar o trabalho com casos reais de vazamento de dados.	Casos incluídos no Capítulo 2; ampliar com fontes revisadas por pares.
27/08/2026	Apresentação da solução	Não nomear o produto antes do capítulo de resultados.	Texto dos Capítulos 4 a 7 usa descrição genérica.

Fonte: Elaborado pela equipe (2026).

6.5 Registro das Decisões

As decisões técnicas relevantes são registradas no formato de *architecture decision records* simplificados: contexto e decisão, alternativas e justificativa, e consequências. Cada decisão aponta para o *commit* em que se tornou observável no código. As justificativas foram reconstruídas a partir do código, do histórico e da documentação do projeto.

Tabela 13 – Registro de decisões técnicas

ID	**Decisão**	**Alternativas e justificativa**	**Consequências**
D01	Interceptar o tráfego por proxy com terminação TLS, em vez de captura passiva de pacotes ou regras de *firewall*.	A captura passiva (testada em I0) observa apenas tráfego cifrado e não permite ler nem alterar o corpo; regras de *firewall* só permitem bloquear. O proxy com terminação TLS é a única das alternativas que permite detectar e redigir o conteúdo.	Exige instalar e confiar numa CA local (D04) e não funciona com clientes que fixam certificados.

ID	Decisão	Alternativas e justificativa	Consequências
D02	Implementar em Go.	Alternativas: linguagens com máquina virtual ou interpretadas. Go entrega binário estático multiplataforma, com biblioteca padrão para HTTP, TLS e X.509.	Distribuição simples (RNF07); dependência de biblioteca de proxy de terceiros.
D03	Detectar dados sensíveis por expressões regulares e valores literais, sem classificadores de aprendizado de máquina.	Os trabalhos revisados usam modelos para reduzir falsos positivos em artefatos estáticos (RAHMAN et al., 2025; TSEBAK; BANAKH; PISKOZUB, 2025); no caminho crítico de cada requisição, a abordagem determinística tem custo previsível, sem dependências nem inferência, e é auditável.	Cobertura limitada aos formatos conhecidos ou configurados; a evolução natural é um estágio híbrido, como o proposto nesses trabalhos.
D04	Gerar uma CA exclusiva na máquina do usuário (RSA 4096, 10 anos), instalá-la no repositório do sistema e nunca versioná-la.	Alternativas: CA compartilhada distribuída com o produto (rejeitada: a chave comprometida afetaria todos os usuários) ou desativar a verificação de certificados nos clientes (rejeitada: reduz a segurança).	O usuário concede confiança à CA; a chave local torna-se ativo crítico (RNF04).
D05	Configuração por arquivo YAML opcional, com padrões embutidos ativos por omissão.	Alternativas: *flags* apenas ou banco de dados. O YAML é legível, versionável e editável por humanos; o funcionamento sem arquivo reduz a barreira de adoção.	Sem recarga a quente: mudanças valem na próxima inicialização.
D06	Persistir a auditoria em arquivo JSON Lines, sem SGBD.	Alternativas: banco embarcado ou servidor. O arquivo de inclusão atende ao requisito de binário único e é consumível por ferramentas externas (Seção 5.3).	Consultas analíticas exigem ferramentas externas; proposta relacional na Seção 5.3.4.

ID	Decisão	Alternativas e justificativa	Consequências
D07	Identificar requisições de IA pelo conteúdo do corpo, e oferecer os modos *whitelist* e *passive*.	Alternativa: lista fixa de *hosts* dos provedores (rejeitada: não cobre novas ferramentas e provedores). A identificação por corpo sustenta o agnosticismo (RNF11); o modo *whitelist* reduz o escopo quando se conhecem os destinos.	Possibilidade de falsos positivos de classificação em corpos JSON com campos homônimos.
D08	Oferecer modo *dry-run*.	Permite validar padrões e calibrar a seletividade sem arriscar a integridade das requisições, mitigando o custo de adoção.	Requer comparar registros com e sem alteração.
D09	Registrar auditoria e eventos de interface de forma assíncrona e não bloqueante, descartando eventos quando a fila está cheia.	Alternativa: gravação síncrona (rejeitada: acoplaria a latência da requisição ao disco). A prioridade é não degradar a experiência do desenvolvedor (RNF09).	Possível perda de registros sob sobrecarga; condição de corrida identificada no registrador (Seção 7.4).
D10	Restringir a negociação TLS interceptada ao HTTP/1.1.	Evita o tratamento de HTTP/2 no *pipeline*, mantendo o processamento restrito ao protocolo suportado pelo proxy.	Clientes que exijam HTTP/2 serão rebaixados ao negociar.
D11	Prover TUI com a biblioteca Bubble Tea.	Alternativas: interface web local ou apenas linha de comando. A TUI mantém o produto em um único binário e no terminal em que a ferramenta já é usada.	Requer terminal com largura mínima de 80 colunas (RNF08).
D12	Distribuir binários por *workflow* de CI, com *checksums*, e por Homebrew e *script* PowerShell.	Mitiga o risco de comprometimento do binário (Seção 5.2) e simplifica a instalação por plataforma.	Manutenção de múltiplos canais de distribuição.

Fonte: Elaborado pela equipe (2026).

6.6 Atualizações da Documentação

A documentação evoluiu junto com o código: 36 dos 64 *commits* alteraram o arquivo principal de documentação, e 23 mensagens de *commit* referem-se explicitamente a documentação. Em duas ocasiões (*commits* f2097ea e faebae9) foram removidos do-

cumentos temporários e desatualizados, o que reduziu a ambiguidade. A Tabela 14 lista os documentos mantidos e sua função.

Tabela 14 – Documentação mantida no repositório

Documento	**Conteúdo**
Documento principal	Instalação (Homebrew e binários), compilação, configuração da CA, comandos, modos do proxy, padrões, desempenho, auditoria, testes e *roadmap*.
Política de segurança	Procedimento de relato de vulnerabilidades.
Guia de contribuição	Pré-requisitos, estratégia de ramos, diretrizes de código, testes e processo de *pull request*.
Exemplo de configuração	Arquivo YAML comentado com todos os campos.
Esquema de auditoria	Descrição campo a campo do registro de auditoria.
Guia de execução de testes	Comandos para cada teste automatizado.
Roteiro de testes manuais	Cenários de validação ponta a ponta com ferramenta de IA.

Fonte: Elaborado pela equipe (2026).

A revisão realizada para esta entrega identificou divergências entre a documentação e o código, descritas na Seção 7.4 (itens P10 a P14). Elas constituem a lista de atualizações pendentes da documentação: corrigir o esquema de auditoria, harmonizar a versão da linguagem, atualizar o roteiro de testes manuais (que cita caminho de máquina específico e comportamento de *streaming* não implementado), revisar a tabela de padrões suportados e a descrição do provedor inferido, e alinhar o guia de contribuição aos ramos existentes.

7 Testes e Validação

Este capítulo apresenta o planejamento dos testes, os casos de teste, a execução com o registro das evidências, os problemas encontrados e o plano de validação com usuários. A verificação confronta o comportamento observado do mecanismo com os requisitos do Capítulo 4; a validação, que avalia se o mecanismo é útil e aceitável para quem o usa, é planejada na Seção 8.2.5.

7.1 Planejamento dos Testes

7.1.1 Objetivos e níveis

Os testes têm três objetivos: (i) verificar os requisitos funcionais críticos (RF01 a RF17) e os requisitos não funcionais mensuráveis (RNF01 a RNF03, RNF09); (ii) obter evidências de desempenho que sustentem a avaliação de impacto operacional, que é uma das contribuições da pesquisa; e (iii) identificar defeitos antes da etapa de experimentação. Foram definidos quatro níveis:

N1 – Unitário automatizado. Testes com o pacote de testes da linguagem sobre o analisador, o filtro, a inferência de provedor, a configuração e a interface de terminal.

N2 – Desempenho. Medições de latência e de vazão do analisador, com cargas de tamanhos variados.

N3 – Integração e sistema. Execução do proxy completo, com cliente e destino reais ou simulados, exercitando o *pipeline* de ponta a ponta e a trilha de auditoria.

7.1.2 Ambiente, dados e critérios

Ambiente de verificação. A execução registrada neste capítulo foi realizada em 14 de outubro de 2026, sobre a versão do código correspondente ao *commit* faebae9, em ambiente Linux (Ubuntu 24.04, amd64), compilador Go 1.24.13 e processador Intel Xeon de 2,1 GHz. Como o módulo declara a versão 1.25 da linguagem, a verificação foi feita em uma cópia descartável com a diretiva ajustada para 1.24, e as dependências foram resolvidas a partir das versões fixadas no módulo; o código-fonte do repositório não foi modificado. A interface de terminal (N1) não pôde ser executada por exigir bibliotecas que não foram resolvidas nesse ambiente.

Dados de teste. Todos os segredos usados são fictícios, com o formato dos *tokens* reais e sem validade. Nenhum teste utiliza credenciais ou código de terceiros.

Critérios de aceitação. (i) todos os testes automatizados aprovados; (ii) nenhum falso positivo nos casos benignos (RNF03); (iii) latência média do analisador inferior a 3 ms para corpos de até 1 MiB (RNF01); (iv) requisição redigida mantida íntegra (JSON válido e tamanho declarado coerente); (v) nenhum defeito de severidade alta em aberto ao final da etapa.

Riscos do plano. A cobertura de testes automatizados do pacote central é parcial (Seção 7.3); o comportamento com ferramentas de IA reais e com HTTPS depende de testes manuais por plataforma; e o desempenho medido em um equipamento não se generaliza para outros.

7.2 Casos de Teste

7.2.1 Testes automatizados existentes

A Tabela 15 agrupa os testes automatizados do repositório, associando-os aos requisitos que verificam. A coluna *Testes* indica a quantidade de funções de teste de primeiro nível; os testes parametrizados executam subtestes adicionais.

7.2.2 Casos de teste de integração e de sistema

Os casos da Tabela 16 exercitam o proxy completo. Foram elaborados a partir do roteiro de testes manuais do repositório e automatizados em um teste de integração que inicia o proxy, um destino HTTP simulado que devolve o corpo recebido e um cliente configurado para usar o proxy. Os casos CT-M01 a CT-M07, que dependem de plataforma, de ferramenta real ou de interação humana, permanecem como testes manuais.

7.3 Execução e Registro de Evidências

7.3.1 Procedimento

Os comandos executados são apresentados no Listagem 7.1. A opção de detecção de corridas de dados foi incluída por ser adequada a um sistema com várias *goroutines* (servidor, *workers* de auditoria e interface).

Listing 7.1 – Comandos de execução dos testes

```
go test -v -count=1 ./...                          # suites automatizadas
go test -count=1 -cover ./internal/guardian/       # cobertura do pacote central
go test -race -count=1 ./...                       # deteccao de corridas de dados
go test -run xxx -bench . -benchmem ./internal/guardian/   # benchmarks
```

Tabela 15 – Suítes de teste automatizadas

ID	O que verifica	Testes	Requisitos	Nível
CT-A01	Detecção correta de cada formato de credencial embutido (16 casos).	1	RF10	N1
CT-A02	Ausência de falsos positivos em 32 dados benignos (identificadores, *hashes*, Base64, URLs, nomes de métodos, *tokens* de outros serviços).	1	RNF03	N1
CT-A03	Padrões personalizados por expressão regular (senha de banco, *string* de conexão, JWT, chave privada).	1	RF12	N1
CT-A04	Padrões personalizados literais (tabelas internas, nomes de projetos).	1	RF12	N1
CT-A05	Ausência de falsos positivos em 10 entradas benignas para padrões personalizados.	1	RF12, RNF03	N1
CT-A06	Latência do analisador (7 cargas $\times$ 1000 iterações), falhando se a média ultrapassar 3 ms.	1	RNF01	N2
CT-A07	Vazão do analisador.	1	RNF01	N2
CT-A08	Leitura, escrita e *round-trip* da configuração YAML; padrões, porta por omissão e erro para YAML inválido ou caminho sem permissão.	7	RF11, RF12, RF17	N1
CT-A09	Casamento de curingas de *host*, tipo de conteúdo e função de classificação de provedores.	6	RF06, RF07	N1
CT-A10	Reconhecimento de requisições de IA por formato de provedor, vazias ou inválidas (9 testes) e decisão de processamento por método, tipo, *host* e modo (8 testes).	17	RF05–RF08	N1
CT-A11	Inferência de provedor e modelo a partir do corpo.	7	RF09	N1
CT-A12	Contadores, anel de eventos, telas e navegação da TUI.	21	RF18–RF20	N1

Fonte: Elaborado pela equipe (2026).

7.3.2 Resultados dos testes automatizados

O pacote central (analisador, filtro, provedor, configuração e auditoria) executou 44 testes de primeiro nível e 52 subtestes, todos aprovados, com cobertura de 31,8% das instruções; o detector de corridas, aplicado à suíte existente, não reportou ocorrências. A análise estática do pacote (`go vet`) não emitiu avisos. Os casos CT-A02 e CT-A05

resultaram em 0 falso positivo em 32 e em 10 entradas, respectivamente. A baixa cobertura decorre de o manipulador de requisições do proxy, a auditoria e os pacotes de CA e de diagnóstico não possuírem testes automatizados; esse é o principal vazio do plano (Tabela 21).

7.3.3 Resultados dos testes de integração

A Tabela 17 apresenta os resultados dos casos CT-I01 a CT-I10, obtidos com o proxy completo e um destino simulado.

7.3.4 Resultados de desempenho

O teste de latência do repositório, executado no ambiente de verificação, resultou em média de 0,95 µs, P95 de 1,92 µs e P99 de 5,06 µs, valores da mesma ordem dos registrados na documentação do projeto (média de 1,07 µs), obtidos em outro equipamento. No entanto, esse teste usa sete cargas de menos de cem bytes, muito menores que as requisições reais de assistentes de código, que carregam contexto de arquivos. Por isso foram realizadas medições complementares com corpos de 1 KB a 5 MiB, contendo código-fonte comum e os nove padrões embutidos ativos (Tabela 18). Cada valor é a mediana de três execuções de *benchmark*.

A análise cresce de forma aproximadamente linear com o tamanho do corpo, a cerca de 2,7 ms por MiB. Para corpos de até 1 MiB, o critério de RNF01 (média inferior a 3 ms) é atendido; no limite de 5 MiB, a análise leva cerca de 13,5 ms, valor ainda pequeno diante do tempo de resposta de um LLM, que costuma ser de segundos, mas acima do limiar do teste automatizado. A vazão de aproximadamente 370 a 475 MB/s indica que o analisador não é o gargalo do *pipeline*. Duas ressalvas metodológicas sobre as medições do repositório são tratadas como problema P09.

7.3.5 Evidências adicionais documentadas no repositório

O repositório documenta, com capturas de tela, resultados de execução em um equipamento com processador Intel Core i5-13400 (Linux): detecção de todos os formatos de credencial, ausência de falsos positivos nos casos benignos, e resultados dos padrões personalizados (cinco de cinco para senha de banco, *string* de conexão e chave privada; um de um para JWT; cinco de cinco para os literais de tabelas e de projetos).

7.3.6 Modelo de registro de evidências

Para as execuções futuras, cada rodada é registrada com os campos da Tabela 19. As execuções de 14/10/2026 correspondem à primeira linha.

7.4 Problemas Identificados e Correções

A verificação e a revisão do código e da documentação identificaram os problemas da Tabela 20. A coluna *Origem* distingue o que foi **confirmado por execução** do que foi identificado por **leitura do código**. A severidade é uma avaliação do autor, baseada no impacto sobre os requisitos. Todos permanecem em aberto na data desta entrega; as correções são o objeto do próximo incremento e cada uma deve gerar ao menos um caso de teste de regressão.

Tabela 20 – Problemas identificados

ID	Descrição	Origem	Severidade	Correção proposta	Req.
P01	Condição de corrida no registrador de auditoria: cada gravação dispara outra *goroutine* que lê e escreve a lista de entradas e o arquivo sem exclusão mútua, comprometendo a integridade da trilha.	Execução (detector de corridas)	Alta	Gravar sob a mesma exclusão mútua, ou usar escritor único com canal. Teste concorrente com `-race`.	RF15, RNF09
P02	Corpos acima de 5 MiB são lidos até o limite e a requisição é encaminhada truncada, produzindo requisição inválida.	Execução (CT-I09)	Alta	Encaminhar o corpo integral sem análise, ou rejeitar explicitamente. Teste de limite.	RF05, RNF02
P03	Durações são armazenadas em milissegundos inteiros; análises que levam microssegundos são registradas como zero, impedindo medir o custo a partir da trilha.	Execução (trilha)	Média	Registrar em microssegundos.	RNF01, RF15
P07	O diagnóstico considera uma variável de ambiente de porta que o proxy não utiliza (a porta vem do YAML), podendo reportar porta diferente da efetiva.	Leitura de código	Baixa	Unificar a fonte da porta.	RF21
P08	Edições feitas na TUI são gravadas no YAML, mas não são aplicadas ao proxy em execução.	Leitura de código	Baixa	Recarga da configuração ou aviso explícito de reinício.	RF19, RF20

Fonte: Elaborado pela equipe (2026).

7.5 Rastreabilidade entre Requisitos e Testes

A Tabela 21 relaciona cada requisito funcional aos casos que o verificam. Requisitos sem cobertura automatizada concentram-se nos componentes de CA, auditoria, diagnóstico, encerramento e distribuição, e serão tratados por novos casos no próximo incremento.

7.6 Síntese

A verificação confirmou o funcionamento do núcleo de detecção e redação: os formatos de credencial suportados são detectados, não houve falsos positivos nos casos de teste, e a requisição redigida permanece válida. O custo de análise é baixo (da ordem de 2,4 ms por MiB) e não representa gargalo frente ao tempo de resposta de um LLM. Por outro lado, a execução revelou dois defeitos de severidade alta (P01 e P02) e limitações de medição (P03, P04 e P09) que afetam diretamente a avaliação de impacto operacional da pesquisa. Esses pontos definem o escopo do próximo incremento antes da etapa de experimentação.

Tabela 16 – Casos de teste de integração e de sistema

ID	Cenário	Resultado esperado	Requisitos
CT-I01	Requisição de IA com chave de projeto e chave de acesso AWS fictícias no *prompt*.	Destino recebe as chaves substituídas pelo rótulo; JSON válido; tamanho declarado coerente.	RF01, RF08, RF10, RF13
CT-I02	Requisição de IA sem dado sensível.	Corpo entregue idêntico ao enviado.	RF13, RNF03
CT-I03	JSON sem campos de requisição de IA contendo chave fictícia.	Requisição ignorada e entregue sem alteração (comportamento por projeto).	RF08
CT-I04	Requisição `POST` com tipo de conteúdo texto simples.	Requisição ignorada e entregue sem alteração.	RF05
CT-I05	Modo *dry-run* com chave fictícia.	Corpo idêntico ao original; evento registrado como detecção.	RF14
CT-I06	*Host* presente na lista de ignorados.	Corpo idêntico ao original.	RF07
CT-I07	Modo *whitelist* com *host* fora da lista de permitidos.	Corpo idêntico ao original.	RF06
CT-I08	Destino inalcançável.	Erro explícito devolvido ao cliente, sem travamento do proxy.	RF16, RNF09
CT-I09	Corpo de 6 MiB (acima do limite de análise).	Requisição encaminhada íntegra ou rejeitada de forma explícita.	RF05, RNF02
CT-I10	100 requisições concorrentes com chave fictícia.	Todas atendidas e uma entrada de auditoria por requisição; trilha sem segredos.	RNF09, RNF10, RF15
CT-M01	Instalação da CA em macOS, Linux e Windows.	CA confiável pelo sistema; diagnóstico indica “instalado”.	RF02, RF03
CT-M02	Ferramenta de IA real configurada para o proxy, com HTTPS.	Funcionamento normal da ferramenta e redação observada na auditoria.	RF04, RNF11
CT-M03	Uso da TUI: painel, criação e exclusão de padrão, edição do proxy.	Telas navegáveis; alterações gravadas no arquivo.	RF18–RF20
CT-M04	Execução de `doctor`, `version` e `help`.	Saídas conforme o esperado.	RF21, RF22
CT-M05	Encerramento por sinal durante tráfego.	Registro de auditoria persistido sem corrupção.	RF23
CT-M06	Conexão mantida aberta além do limite de tempo.	Conexão encerrada de forma controlada.	RNF09
CT-M07	Instalação por Homebrew e por binário, com verificação de *checksum*.	Binário executável; *checksum* confere.	RF24, RNF07

Fonte: Elaborado pela equipe (2026), a partir do roteiro de testes do repositório.

Tabela 17 – Resultados dos casos de integração (14/10/2026)

ID	Observação	Resultado
CT-I01	Ambas as chaves foram substituídas pelo rótulo; o JSON permaneceu válido e o tamanho declarado coincidiu com o corpo entregue.	Aprovado
CT-I02	Corpo entregue idêntico.	Aprovado
CT-I03	Corpo entregue sem alteração; a chave fictícia **não** foi redigida por o JSON não conter campos de IA.	Conforme o projeto; limitação
CT-I04	Corpo entregue sem alteração.	Aprovado
CT-I05	Corpo idêntico ao original; evento registrado com as detecções.	Aprovado
CT-I06	Corpo idêntico.	Aprovado
CT-I07	Corpo idêntico.	Aprovado
CT-I08	O cliente recebeu resposta de erro (status 500), e o proxy continuou operante.	Aprovado, com ressalva
CT-I09	Foram enviados 6.291.516 bytes e o destino recebeu 5.242.880 (exatamente 5 MiB), com JSON inválido: a requisição é truncada (P02).	**Reprovado**
CT-I10	99 das 100 requisições concorrentes foram atendidas; a trilha registrou 108 das 109 entradas esperadas no cenário completo; nenhuma entrada contém segredo. A causa da falha isolada não foi investigada. Com o detector de corridas, o teste de integração acusou corridas de dados.	**Reprovado**

Fonte: Elaborado pela equipe (2026).

Tabela 18 – Latência do analisador por tamanho de corpo (ambiente de verificação)

Carga	Tempo por operação	Vazão	Memória por operação
1 KB	2,4 µs	472 MB/s	1,1 KB
10 KB	22,7 µs	451 MB/s	10,6 KB
100 KB	0,22 ms	474 MB/s	104 KB
1 MiB	2,44 ms	429 MB/s	1,0 MiB
5 MiB (limite de análise)	13,5 ms	388 MB/s	5,0 MiB
100 KB, com duas credenciais a redigir	0,28 ms	371 MB/s	314 KB (19 alocações)

Fonte: Elaborado pela equipe (2026).

Tabela 19 – Modelo de registro de execução de testes

Data	Versão	Ambiente	Casos	Resultado e evidência
14/10/2026	faebae9	Linux amd64, Go 1.24.13, Xeon 2,1 GHz	CT-A01 a A11; CT-I01 a I10; *benchmarks*	44 testes aprovados; 2 casos de integração reprovados; saída de teste arquivada

Fonte: Elaborado pelo autor (2026).

Tabela 21 – Matriz de rastreabilidade requisitos–testes

Req.	**Casos**	**Req.**	**Casos**
RF01	CT-I01 a I10	RF14	CT-I05
RF02, RF03	CT-M01 (não executado)	RF15	CT-I10 (inspeção da trilha)
RF04	CT-M02 (não executado)	RF16	CT-I08
RF05	CT-A10, CT-I04, CT-I09	RF17	CT-A08
RF06	CT-A10, CT-I07	RF18–RF20	CT-A12 (não executado), CT-M03
RF07	CT-A09, CT-A10, CT-I06	RF21, RF22	CT-M04 (não executado)
RF08	CT-A10, CT-I03	RF23	CT-M05 (não executado)
RF09	CT-A11	RF24	CT-M07 (não executado)
RF10	CT-A01, CT-A02	RF25	—
RF11, RF12	CT-A03 a A05, CT-A08	RNF01	CT-A06, CT-A07, *benchmarks*
RF13	CT-I01, CT-I02	RNF03	CT-A02, CT-A05, CT-I02

Fonte: Elaborado pela equipe (2026).

8 Resultados e Discussão

Este capítulo apresenta o produto resultante do trabalho, descreve os resultados obtidos na verificação (Capítulo 7), compara-os com os objetivos definidos no Capítulo 1, discute-os criticamente e registra as limitações do estudo. Uma advertência de escopo orienta a leitura: os resultados são de *verificação do artefato*, obtidos em 14 de outubro de 2026 sobre o *commit* faebae9, em ambiente Linux e com destino simulado. Eles não incluem validação com usuários nem execução com ferramentas de IA reais sobre HTTPS (Seção 8.2.5), o que condiciona as conclusões da Seção 8.3 e do Capítulo 9.

8.1 Apresentação do Produto

O mecanismo especificado e projetado nos Capítulos 4 e 5, descrito até aqui sem nome, conforme a orientação registrada na Tabela 12, é implementado pelo **GalileuCLI**: uma ferramenta de linha de comando, de código aberto, que atua como *proxy* entre as ferramentas de desenvolvimento assistidas por LLM e os provedores externos de IA, detectando e redigindo dados sensíveis no tráfego de saída. O executável chama-se `galileu`. A Tabela 22 resume suas características.

Tabela 22 – Ficha técnica do GalileuCLI

Item	**Descrição**
Finalidade	*Proxy* de segurança e governança que detecta e redige dados sensíveis em requisições enviadas a provedores de LLM, com trilha de auditoria.
Repositório	Público, no GitHub: <https://github.com/eubrunocase/GalileuCLI> (acesso em 26 out. 2026).
Versão	Última *release* publicada: v1.0.1. A verificação do Capítulo 7 foi feita sobre o *commit* faebae9, dois *commits* posteriores a essa *release*.
Linguagem e dimensão	Go; cerca de 3.600 linhas de código de produção e 1.300 de testes; 64 *commits* entre 16/04 e 17/07/2026 (Seção 6.3).
Plataformas	macOS (Intel e Apple Silicon), Linux (amd64) e Windows (amd64).
Distribuição	Binários de *release* com *checksums* SHA-256; Homebrew (macOS e Linux); *script* PowerShell (Windows).
Licença	Apache 2.0.

Fonte: Elaborado pela equipe (2026), a partir do repositório do projeto.

8.1.1 Como o produto é usado

O fluxo de uso previsto tem cinco passos. O usuário instala o binário (por exemplo, com o Homebrew) e executa o diagnóstico do ambiente (`galileu doctor`). Em seguida, inicia o *proxy*, com ou sem a interface de terminal (`galileu --tui`); na primeira execução, o produto gera a autoridade certificadora local e tenta instalá-la no sistema operacional, o que exige privilégios. O terceiro passo é apontar a ferramenta de IA para o *proxy*, que escuta por padrão na porta 9000. A partir daí o desenvolvedor trabalha normalmente: cada requisição de IA é analisada e as credenciais reconhecidas são substituídas pelo marcador `[REDACTED_BY_GALILEU]` antes do encaminhamento ao provedor. Por fim, o responsável pela governança consulta o arquivo de auditoria. O modo `--dry-run` permite executar todo o fluxo apenas detectando e registrando, sem alterar as requisições, o que serve para calibrar os padrões antes de habilitar a redação.

A Figura 11 mostra a inicialização do produto no terminal. A Listagem 8.1 ilustra o efeito sobre uma requisição, construída a partir do comportamento verificado no caso CT-I01 (Tabela 17), com uma chave fictícia; a Listagem 8.2 mostra o registro de auditoria correspondente, com valores ilustrativos e campos selecionados, em que o conteúdo da requisição não aparece.

Figura 11 – Inicialização do GalileuCLI no terminal



Fonte: Elaborado pela equipe (2026), a partir da execução do produto em macOS.

Listing 8.1 – Efeito da redação sobre uma requisição (exemplo ilustrativo)

```
Antes (enviado pela ferramenta):
{"model":"modelo-exemplo","messages":[{"role":"user",
 "content":"Corrija o erro. OPENAI_API_KEY=sk-EXEMPLO00000000000000000FICTICIO"}]}
```

```
4
5 Depois (encaminhado ao provedor):
6 {"model":"modelo-exemplo","messages":[{"role":"user",
7  "content":"Corrija o erro. OPENAI_API_KEY=[REDACTED_BY_GALILEU]"}]}
```

Listing 8.2 – Registro de auditoria correspondente (campos selecionados, valores ilustrativos)

```
1 {"timestamp":"2026-10-14T15:02:11Z","request_id":"a1b2c3","host":"api.openai.com",
2  "provider":"openai","redacted":true,"pattern_count":1,
3  "detected_patterns":["openai_key"],"message_count":1,
4  "request_body_size_bytes":142,"analysis_duration_ms":0,"proxy_error":false}
```

A documentação do repositório demonstra a compatibilidade com diversas ferramentas de IA entre elas OpenCode, Claude Code, Cursor, Windsurf, GitHub Copilot e Gemini CLI, com o modo *passive* recomendado para as que acessam vários *hosts*, e inclui uma demonstração com uma dessas ferramentas(Opencode).

8.2 Resultados Obtidos

8.2.1 Detecção e redação

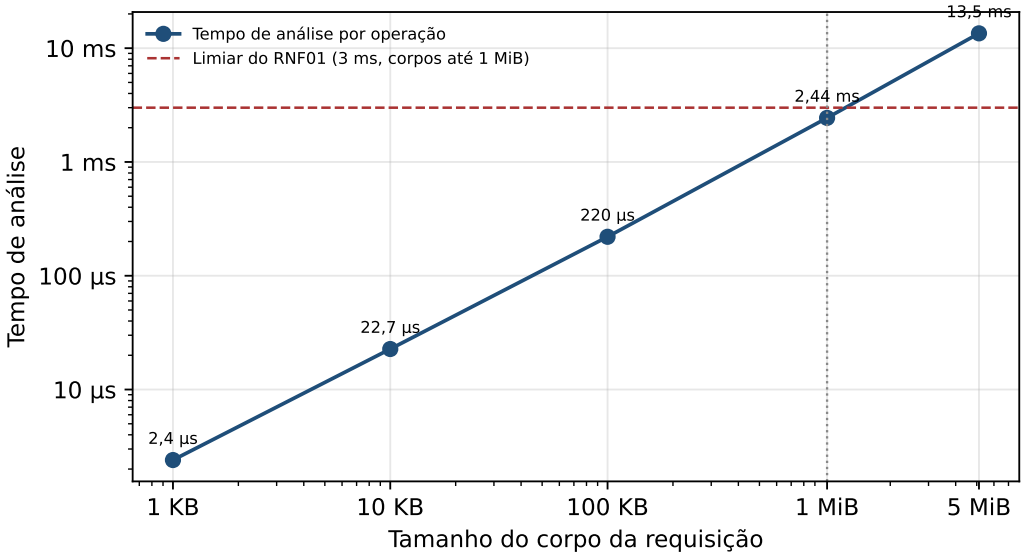
O pacote central do produto executou 44 testes de primeiro nível e 52 subtestes, todos aprovados, com cobertura de 31,8% das instruções; a análise estática (`go vet`) não emitiu avisos e o detector de corridas de dados, aplicado à suíte existente, não reportou ocorrências. Os 16 casos de detecção (CT-A01) foram identificados corretamente e não houve falso positivo em nenhuma das 32 entradas benignas (CT-A02) nem das 10 entradas benignas dos padrões personalizados (CT-A05). Os padrões personalizados por expressão regular e por valor literal foram detectados nos casos previstos (CT-A03 e CT-A04). Esse resultado deve ser lido com uma ressalva: os 16 casos exercitam sete dos nove padrões embutidos: não há caso de teste para o padrão de *token* do Discord nem para o de chave secreta da AWS (Seção 8.2.4).

Na integração com o *proxy* completo e destino simulado (Tabela 17), a requisição de IA com chaves fictícias chegou ao destino com as chaves substituídas, JSON válido e tamanho declarado coerente (CT-I01); a requisição sem dado sensível chegou idêntica (CT-I02); e os casos de filtragem por método, tipo de conteúdo, *host* ignorado e modo *whitelist* (CT-I04, CT-I06 e CT-I07) e o modo *dry-run* (CT-I05) comportaram-se como especificado. Dos dez casos, seis foram aprovados, um foi aprovado com ressalva (CT-I08, em que o erro de conexão foi devolvido com status 500 em requisição HTTP simples), um refletiu uma limitação conhecida do projeto (CT-I03, em que uma chave em JSON sem campos de requisição de IA não é redigida, por decisão D07) e dois foram reprovados (CT-I09 e CT-I10, tratados na Seção 8.2.4).

8.2.2 Desempenho

As medições de latência do analisador (Tabela 18) mostram crescimento aproximadamente linear com o tamanho do corpo, a cerca de 2,7 ms por MiB, com vazão entre 371 e 474 MB/s (Figura 12). Para corpos de até 1 MiB, a média de 2,44 ms atende ao critério de 3 ms do RNF01; no limite de análise de 5 MiB, o tempo é de 13,5 ms, acima do limiar definido para o teste automatizado, mas pequeno diante do tempo de resposta de um LLM, que costuma ser medido em segundos. Para cargas pequenas, como as do teste de latência do repositório, a média foi de 0,95 µs (P95 de 1,92 µs e P99 de 5,06 µs), da mesma ordem do valor documentado pelo projeto em outro equipamento (1,07 µs). A memória alocada por operação é aproximadamente igual ao tamanho do corpo, porque a análise opera sobre uma cópia dele.

Figura 12 – Latência do analisador por tamanho do corpo da requisição



Fonte: Elaborado pela equipe (2026), a partir da Tabela 18 (mediana de três execuções; escalas logarítmicas).

Esses números se referem ao **analisador**, e não ao *proxy* de ponta a ponta. O custo do estabelecimento da conexão TLS, da emissão de certificados e do encaminhamento não foi medido, e a trilha de auditoria não permite estimá-lo, porque registra a duração da análise em milissegundos inteiros e, portanto, grava zero para análises que levam microssegundos (P03).

8.2.3 Situação dos requisitos

A Tabela 23 classifica os 25 requisitos funcionais segundo a evidência obtida na verificação, e a Tabela 24 faz o mesmo para os 14 requisitos não funcionais. A classificação distingue o que foi *verificado* do que apenas está *implementado*: dez requisitos

funcionais não têm evidência na rodada de 14/10/2026, por dependerem de testes manuais por plataforma, de ferramenta real ou de interação humana (CT-M01 a CT-M07), que não foram executados, ou, no caso da interface de terminal, de bibliotecas que não puderam ser resolvidas no ambiente de verificação.

Tabela 23 – Situação de verificação dos requisitos funcionais

Situação	**Requisitos e observação**	**Qtde.**
Verificado	RF06 a RF09, RF11 a RF14, RF16 e RF17: evidência favorável nos testes automatizados e de integração.	10
Parcialmente verificado	RF01 (apenas HTTP com destino simulado, sem HTTPS); RF05 (corpos acima de 5 MiB são truncados, P02); RF10 (sete dos nove padrões testados; ver P15 e P16).	3
Evidência desfavorável	RF15: a trilha de auditoria ficou incompleta e sujeita a corrida de dados no caso CT-I10 (P01).	1
Sem evidência na rodada	RF02, RF03, RF04 (CA e interceptação TLS) e RF18 a RF24 (TUI, diagnóstico, encerramento e distribuição).	10
Não implementado	RF25 (*plugins* de padrões), adiado deliberadamente.	1
	Total	**25**

Fonte: Elaborado pela equipe (2026), a partir da Tabela 21 e dos resultados deste capítulo.

Tabela 24 – Situação de verificação dos requisitos não funcionais

ID	**Situação**	**Evidência**
RNF01	Atendido	Média de 2,44 ms para corpos de 1 MiB (analisador).
RNF02	Não atendido	Corpo de 6 MiB chegou truncado ao destino (CT-I09, P02).
RNF03	Atendido no conjunto de teste	Zero falsos positivos em 42 entradas benignas (CT-A02, CT-A05).
RNF04	Atendido (inspeção)	Chave RSA de 4096 bits, gravada com permissão 0600; arquivos de certificado e chave excluídos do versionamento.
RNF05	Parcial	O esquema da trilha não contém campo para o conteúdo da requisição; porém o nome de padrões literais, que inclui o valor, é propagado à trilha (P17).
RNF06	Sem evidência	Verificação feita apenas em Linux.
RNF07	Sem evidência	Instalação por *script* e por gerenciador de pacotes não testada (CT-M07).
RNF08	Sem evidência	Interface de terminal não executada; sem teste com usuários.
RNF09	Parcial	Erro explícito em destino inalcançável (CT-I08); falhas de integridade da trilha sob concorrência (CT-I10, P01).
RNF10	Parcial	Campos de identificação de requisição, sessão e máquina presentes no esquema; unicidade não verificada; trilha incompleta em CT-I10.

ID	Situação	Evidência
RNF11	Sem evidência	Compatibilidade com ferramentas reais não reexecutada (CT-M02).
RNF12	Atendido	Padrões e modos alteráveis por arquivo, sem recompilação (CT-A08).
RNF13	Parcial	Integração contínua, testes e documentação presentes; cobertura do pacote central de 31,8%.
RNF14	Atendido (inspeção)	Licença Apache 2.0 no repositório.

Fonte: Elaborado pela equipe (2026).

8.2.4 Defeitos e achados adicionais

Os cinco problemas registrados na Tabela 20 (P01, P02, P03, P07 e P08) permanecem sem correção publicada: o ramo principal do repositório continua no *commit* faebae9, o mesmo submetido à verificação. Entre eles, dois têm severidade alta: a condição de corrida no registrador de auditoria (P01), que comprometeu a integridade da trilha na execução com concorrência (CT-I10), e o truncamento de corpos acima de 5 MiB (P02), que entrega ao provedor uma requisição inválida (CT-I09).

A elaboração deste capítulo incluiu uma revisão adicional do código-fonte dos padrões de detecção e do caminho dos nomes de padrões até a trilha de auditoria, que identificou três achados novos, resumidos na Tabela 25. Diferentemente de P01 e P02, eles foram obtidos **por leitura do código**, e não por execução; sua confirmação por testes de regressão faz parte das correções propostas.

8.2.5 Validação com usuários

A validação do produto com usuários, prevista no planejamento (Capítulos 4 e 7), não foi executada neste ciclo. Em consequência, os perfis da Tabela 2 e os requisitos de usabilidade e compatibilidade (RNF08 e RNF11) permanecem sem evidência empírica de uso por terceiros, e as afirmações deste trabalho sobre utilidade e aceitação limitam-se ao que foi observado por quem desenvolveu o artefato.

8.3 Comparação com os Objetivos

A Tabela 26 confronta cada objetivo específico, definidos na Seção 1.3, com as evidências produzidas e classifica seu atendimento.

Tabela 25 – Achados adicionais da revisão do código (*commit* faebae9)

ID	Descrição	Severidade	Correção proposta
P15	O padrão `discord_token` tem a mesma expressão regular do padrão `slack_token` (`xox[baprs]-`). Tokens do Discord, que têm outro formato, não são reconhecidos; um comentário do próprio teste automatizado registra essa diferença. O requisito RF10 menciona o Discord.	Média	Implementar o formato correto, ou retirar o padrão e ajustar RF10 e a documentação; criar caso de teste.
P16	O padrão `aws_secret_key` exige o prefixo `wJalr`, que corresponde à chave secreta de exemplo da documentação da AWS. Chaves secretas reais são geradas aleatoriamente e, em geral, não o apresentam; o padrão tende a não detectá-las. Não há caso de teste para ele.	Alta	Substituir por detecção baseada em contexto (por exemplo, valor de 40 caracteres associado ao nome da variável) e testar com valores aleatórios no formato real.
P17	Para padrões personalizados do tipo literal, o nome do padrão é composto com o próprio valor sensível. Esse nome é registrado em `detected_patterns` na trilha de auditoria e exibido no console, o que contraria o RNF05 (a trilha não deve conter os segredos detectados).	Alta	Usar apenas o nome do padrão e um índice; teste de regressão que verifique a ausência do valor na trilha.

Fonte: Elaborado pela equipe (2026), a partir da leitura do código-fonte.

Tabela 26 – Atendimento dos objetivos específicos

Obj.	Evidência	Situação	Ressalva
OE1	Cinco cadeias de busca; 15 trabalhos únicos analisados e 8 incluídos; lacuna caracterizada em três eixos (Capítulo 2).	Atingido parcialmente	Base única (IEEE Xplore); casos reais sem fonte arbitrada; sem referência normativa;.
OE2	Cinco perfis de usuários, 25 requisitos funcionais e 14 não funcionais, com priorização e critérios de aceitação (Capítulo 4).	Atingido	Perfis não validados com usuários (L02).
OE3	Arquitetura, modelo de dados, interfaces e diagramas de casos de uso, sequência, fluxo e classes (Capítulo 5).	Atingido	—

Obj.	Evidência	Situação	Ressalva
OE4	24 dos 25 requisitos funcionais implementados; 64 *commits* em nove incrementos; 12 decisões registradas; documentação mantida (Capítulo 6).	Atingido	Defeitos abertos (P01, P02, P15 a P17).
OE5	Dezesseis casos de detecção, zero falsos positivos em 42 entradas benignas, redação íntegra em CT-I01 (Seção 8.2).	Atingido parcialmente	Dados sintéticos; sete de nove padrões testados; truncamento em CT-I09; P15 a P17.
OE6	Latência e vazão do analisador de 1 KB a 5 MiB (Tabela 18).	Atingido parcialmente	Apenas o analisador; latência de ponta a ponta e impacto na utilidade não medidos (L04).

Fonte: Elaborado pela equipe (2026).

Três dos seis objetivos foram atingidos integralmente e três, parcialmente. O objetivo geral compreende três verbos: *projetar* e *implementar* foram atendidos (OE2 a OE4); *avaliar* foi atendido de modo parcial (OE5 e OE6), porque a avaliação se restringiu à verificação do artefato em condições controladas. A leitura honesta dos resultados é, portanto, a de que o mecanismo foi construído e que seu núcleo de detecção e redação funciona para os casos testados e a baixo custo de análise, mas que a eficácia em tráfego real e a aceitação por usuários não foram demonstradas.

Quanto à questão de pesquisa RQ5 (Tabela 1), os resultados respondem apenas à parte do custo computacional da análise, que é desprezível frente ao tempo de resposta de um LLM; a latência de ponta a ponta e a perda de utilidade causada pela redação permanecem em aberto.

8.4 Análise Crítica e Discussão

8.4.1 O que os resultados de detecção sustentam e o que não sustentam

O resultado de zero falso positivo e de detecção de todos os casos positivos deve ser interpretado com cuidado. Os casos positivos e negativos foram escritos pelos

próprios desenvolvedores, a partir dos formatos que os padrões foram desenhados para reconhecer; eles medem a *conformidade* da implementação com a especificação, e não a revocação ou a precisão em tráfego real. Os achados P15 e P16 ilustram o limite desse tipo de teste: o conjunto de testes passa integralmente, e ainda assim dois dos nove padrões embutidos não cumprem, na prática, o que o requisito afirma, justamente os dois sem caso de teste.

Há uma razão estrutural para o bom resultado em falsos positivos: os padrões embutidos são ancorados em prefixos de formatos específicos de provedores (como `sk-`, `ghp_` ou `AKIA`), o que torna improvável que identificadores comuns os acionem. A mesma propriedade limita a revocação: credenciais sem prefixo característico, segredos de formato desconhecido, dados pessoais e código proprietário não são detectados, a menos que o usuário os descreva por padrões personalizados. Como a fundamentação teórica definiu o problema de forma ampla (credenciais, dados pessoais e código proprietário), há uma distância entre o problema e a cobertura efetiva do artefato, que o Capítulo 9 retoma.

8.4.2 Comparação com a literatura

Os trabalhos revisados que reportam métricas de detecção usam abordagens baseadas em aprendizado de máquina: Rahman et al. (2025) obtêm F1 de 0,9852 combinando extração por expressões regulares e classificação por modelos de linguagem ajustados, Tsebak, Banakh e Piskozub (2025) reportam redução de falsos positivos ao combinar um modelo contextual a um classificador de *gradient boosting*, e Sakapertana et al. (2025) obtêm 80,96% de acurácia com SVM na detecção de dados pessoais em respostas de API. Esses valores **não são comparáveis** aos deste trabalho: os conjuntos de dados, as tarefas e as métricas diferem, e o presente estudo não calculou precisão, revocação ou F1 sobre um corpus rotulado. O que se pode afirmar é a diferença de abordagem: a decisão D03 trocou cobertura por previsibilidade, com um estágio determinístico, auditável e de custo conhecido por requisição, ao passo que os estudos citados incorporam um estágio de inferência cujo custo de latência, como observado na Seção 2.9, não é reportado.

Nessa mesma direção, Xu et al. (2025) destacam que as técnicas de computação segura impõem custos computacionais e de latência relevantes em cenários interativos; o custo de análise aqui medido, da ordem de milissegundos para corpos de até 1 MiB, situa a abordagem determinística no extremo oposto desse espectro, ao custo de uma proteção mais restrita. A seletividade defendida por Pan e Wu (2025), calibrar a proteção conforme a sensibilidade em vez de aplicá-la de modo uniforme, aparece no mecanismo na ativação individual de padrões e no modo *dry-run*, e foi verificada nos casos CT-A03 a CT-A05 e CT-A08; o que não foi avaliado é o efeito dessa seletividade

sobre a utilidade das respostas do assistente.

8.4.3 Custo operacional

O resultado de desempenho sustenta que o analisador não é o gargalo do fluxo: com vazão acima de 370 MB/s, a análise de um corpo típico de requisição de assistente de código, de até algumas centenas de quilobytes, leva menos de 1 ms. Três ressalvas moderam essa conclusão. Primeiro, o custo de ponta a ponta não foi medido (Seção 8.2). Segundo, a análise copia o corpo e, por isso, o consumo de memória cresce com o tamanho das requisições e com a concorrência; o comportamento sob carga, com muitas requisições grandes simultâneas, não foi medido, e o único teste de concorrência (CT-I10) foi reprovado. Terceiro, as cargas das medições complementares usam código-fonte comum, de modo que corpos com estrutura muito diferente (por exemplo, dados codificados em Base64) podem comportar-se de outra forma.

8.4.4 Decisões de projeto e suas consequências

Os resultados permitem avaliar três decisões registradas na Tabela 13. A decisão D07, de identificar requisições de IA pelo conteúdo do corpo, sustenta a independência de ferramenta e provedor, mas tem como custo que um segredo enviado em JSON sem os campos reconhecidos não é redigido (CT-I03); trata-se de uma escolha de escopo, que delimita a análise às requisições reconhecidamente de IA, ao custo de uma lacuna de proteção que o usuário precisa conhecer. A decisão D09, de registrar a auditoria de forma assíncrona e não bloqueante, protege a latência, mas está na origem da condição de corrida P01 e da possibilidade de perda de registros sob sobrecarga: a prioridade dada à experiência do desenvolvedor custou integridade da evidência, o que é uma tensão real entre os requisitos de desempenho e de governança. Já a decisão D04, de gerar uma CA exclusiva por máquina, mostrou-se adequada do ponto de vista de segurança (RNF04), mas concentra o risco do produto em um ativo, a chave da CA, que o usuário precisa proteger.

O achado P17 evidencia uma tensão semelhante entre conveniência e privacidade: um nome de padrão que inclui o valor facilita a leitura do relatório, mas transforma a trilha, concebida para ser livre de segredos, em um local onde eles podem ser gravados.

8.4.5 Governança e auditabilidade

O mecanismo cumpre parte da função de governança descrita na fundamentação teórica: a política sobre o que não deve sair é aplicada de forma técnica e deixa um registro do que foi detectado, em quais destinos e com qual padrão, sem armazenar o

conteúdo. Entretanto, a capacidade de *produzir* evidência não equivale à de *garantir* evidência. A trilha não é protegida contra adulteração, reside no mesmo equipamento que gera os eventos, pode perder registros sob carga (P01) e, para padrões literais, pode conter o próprio segredo (P17). Além disso, o mapeamento entre os padrões do produto e as categorias de dados de normas como a LGPD ou a ISO/IEC 27701 não foi feito, e a lacuna normativa registrada na fundamentação teórica permanece.

8.4.6 Ameaças à validade

Quanto à validade de construto, os testes foram escritos pelos próprios desenvolvedores, e a latência foi medida no analisador, não no fluxo completo. Quanto à validade interna, as medições de desempenho usam a mediana de três execuções, um número pequeno, e o ambiente (um processador Intel Xeon de 2,1 GHz) pode ter variações não controladas; o caso CT-I10 revelou comportamento não determinístico cuja causa não foi investigada. Quanto à validade externa, os resultados vêm de um único ambiente, com um destino simulado, e não se generalizam para outras plataformas, para HTTPS sobre ferramentas reais nem para o perfil de uso de equipes. A comparação com a literatura é qualitativa, pelas razões expostas acima.

8.5 Limitações

A Tabela 27 consolida as limitações do estudo e do produto, com o seu efeito sobre os resultados e o tratamento proposto. As limitações L01 a L06 são do **estudo** (como se avaliou); as L07 a L12 são do **produto** (o que ele faz e não faz).

Tabela 27 – Limitações do estudo e do produto

ID	**Limitação**	**Efeito sobre os resultados**	**Tratamento**
L01	Conjunto de teste sintético e escrito pela equipe (16 casos positivos e 42 negativos).	Mede conformidade, não precisão e revocação em tráfego real (OE5).	Corpus rotulado e cálculo de precisão, revocação e F1.
L02	Ausência de validação com usuários.	Usabilidade, aceitação e utilidade não avaliadas (RNF08, RNF11).	Estudo com desenvolvedores (Seção 9.5).
L03	Verificação em um único ambiente (Linux, destino HTTP simulado); casos manuais CT-M01 a CT-M07 não executados.	Dez requisitos funcionais sem evidência; HTTPS e demais plataformas não verificados.	Execução do roteiro manual em três plataformas.

ID	Limitação	Efeito sobre os resultados	Tratamento
L04	Latência medida apenas no analisador; impacto da redação sobre a utilidade das respostas não medido.	OE6 e RQ5 atendidos parcialmente.	Medição de ponta a ponta; comparação de respostas com e sem redação.
L05	Cobertura de testes de 31,8% no pacote central; sem testes do manipulador de requisições, da auditoria, da CA e do diagnóstico.	Defeitos como P01, P02 e P15 a P17 escaparam dos testes existentes.	Testes de regressão para cada defeito.
L06	Revisão da literatura restrita ao IEEE Xplore e a oito estudos; casos reais sem fonte arbitrada; sem referência normativa.	OE1 atendido parcialmente; justificativa e governança menos sustentadas.	Ampliar bases e fontes (Seção 9.5).
L07	Detecção por padrões de credenciais de formato conhecido; sem detecção de código proprietário, dados pessoais ou segredos sem formato característico.	Cobertura efetiva menor que o problema definido; P15 e P16 a reduzem ainda mais.	Detecção híbrida e padrões de dados pessoais.
L08	Somente corpos de requisições de saída, *POST* ou *PUT*, JSON ou formulário, com campos de IA e até 5 MiB são analisados; respostas não são inspecionadas.	Segredos fora desse perfil (CT-I03, CT-I09) não são protegidos.	Ampliar tipos de conteúdo; tratar corpos grandes (P02).
L09	Interceptação depende de CA local confiável; clientes com fixação de certificado não funcionam; negociação restrita ao HTTP/1.1.	Ferramentas incompatíveis ficam sem proteção; a chave da CA é ativo crítico.	Documentar; avaliar HTTP/2.
L10	Defeitos de severidade alta em aberto: P01, P02, P16 e P17.	Integridade da trilha, das requisições grandes e a eficácia para AWS e padrões literais comprometidas.	Correção e teste de regressão.
L11	Configuração sem recarga a quente (P08); sem sistema de *plugins* (RF25); sem SGBD.	Operação e análise da trilha menos práticas.	Recarga de configuração; banco embarcado.
L12	Trilha de auditoria local, sem proteção de integridade.	Evidência não é à prova de adulteração.	Encadeamento de *hashes* ou assinatura.

Fonte: Elaborado pela equipe (2026).

9 Conclusões

Este capítulo retoma o problema que motivou o trabalho, avalia o atendimento dos objetivos, apresenta as contribuições e as limitações e propõe caminhos para trabalhos futuros.

9.1 Retomada do Problema

O uso de assistentes de programação baseados em LLM faz com que o contexto do desenvolvedor, incluindo credenciais, *tokens* e arquivos de configuração, seja enviado a provedores externos de IA como efeito colateral de um uso legítimo, o que escapa aos controles perimetrais tradicionais (Capítulo 1). A revisão da literatura mostrou que as soluções existentes avaliam a detecção sobre artefatos estáticos, atuam no treinamento ou na arquitetura interna do agente, ou substituem o provedor externo por um modelo local, e que nenhum dos estudos analisados combina a interceptação do tráfego em tempo real, a independência de ferramenta e de provedor e a avaliação do impacto operacional (Capítulo 2). O trabalho respondeu a essa lacuna com o projeto, a implementação e a verificação de um *proxy* de segurança, o GalileuCLI.

9.2 Avaliação dos Objetivos

O objetivo geral, de projetar, implementar e avaliar o mecanismo, foi atingido de modo parcial. O mecanismo foi projetado e implementado: 24 dos 25 requisitos funcionais estão implementados, com arquitetura, modelo de dados e decisões documentados (OE2 a OE4). A avaliação, por sua vez, restringiu-se à verificação do artefato em condições controladas (OE5 e OE6), conforme a Tabela 26.

Os resultados permitem afirmar que, para as credenciais de formato conhecido e para os padrões configurados pelo usuário, a detecção e a redação funcionam nos casos testados, preservam a validade da requisição e não produziram falsos positivos no conjunto de teste, e que o custo de análise é da ordem de 2,7 ms por MiB, desprezível frente ao tempo de resposta de um LLM. Não permitem afirmar que o mecanismo seja eficaz em tráfego real, que funcione com ferramentas de IA reais sobre HTTPS em todas as plataformas, que o custo de ponta a ponta seja igualmente baixo ou que os usuários o considerem útil e aceitável. Quanto às questões de pesquisa, RQ1 a RQ4 foram respondidas pela revisão da literatura, e RQ5 apenas na dimensão do custo computacional da análise.

9.3 Contribuições

- **Artefato:** um *proxy* de código aberto (Apache 2.0), distribuído como binário único para macOS, Linux e Windows, que detecta e redige credenciais no tráfego de ferramentas de IA, com modo de simulação, interface de terminal e trilha de auditoria sem conteúdo.
- **Caracterização da lacuna:** a síntese da literatura em três eixos (interceptação em tempo real, independência de ferramenta e provedor, avaliação do impacto operacional) e o posicionamento do mecanismo nesse espaço.
- **Especificação rastreável:** cinco perfis de usuários, 25 requisitos funcionais e 14 não funcionais com critérios de aceitação, matriz de rastreabilidade com os testes e registro de 12 decisões técnicas.
- **Evidência de desempenho:** medições de latência e vazão do analisador para corpos de 1 KB a 5 MiB, que não são reportadas nos estudos revisados.
- **Registro crítico de defeitos e limitações:** um catálogo de problemas, alguns de severidade alta, e a constatação de que testes escritos pelos próprios desenvolvedores não detectaram falhas de cobertura em padrões de detecção.

9.4 Limitações

As limitações detalhadas na Seção 8.5 condicionam todas as conclusões acima. As principais são o conjunto de teste sintético, a ausência de validação com usuários, a verificação em um único ambiente com destino simulado, a medição de latência restrita ao analisador, a cobertura de detecção limitada a credenciais de formato conhecido e a presença de defeitos de severidade alta ainda sem correção (P01, P02, P16 e P17).

9.5 Recomendações para Trabalhos Futuros

Em ordem de prioridade:

1. **Corrigir e consolidar.** Corrigir os defeitos P01, P02, P03 e P15 a P17, acompanhando cada correção de um teste de regressão, e ampliar a cobertura de testes do manipulador de requisições, da auditoria, da CA e do diagnóstico.
2. **Avaliar com rigor.** (i) Construir um corpus rotulado e calcular precisão, revocação e F1; (ii) medir a latência de ponta a ponta com ferramentas reais, sobre HTTPS, nas três plataformas; (iii) realizar a validação com desenvolvedores, com

tarefas definidas e instrumento padronizado de usabilidade; (iv) comparar as respostas do LLM com e sem redação, para medir a perda de utilidade; (v) testar o comportamento sob carga e uso de memória.

3. **Ampliar a detecção.** Incorporar um estágio híbrido, com classificador contextual sobre os candidatos extraídos por padrões, em linha com Rahman et al. (2025) e Tsebak, Banakh e Piskozub (2025), medindo seu custo de latência; adicionar padrões de dados pessoais; tratar outros tipos de conteúdo, respostas dos provedores e HTTP/2.

4. **Fortalecer a governança.** Proteger a integridade da trilha (encadeamento de *hashes* ou assinatura), adotar armazenamento consultável embarcado, permitir a recarga de configuração, implementar o sistema de *plugins* (RF25) e mapear as categorias de dados detectadas para normas como a LGPD e a ISO/IEC 27701.

5. **Fortalecer a fundamentação.** Ampliar a revisão para outras bases (como SciELO e anais da SBC), incluir fontes arbitradas sobre casos reais de vazamento e estudos sobre arquiteturas intermediárias.

9.6 Considerações Finais

O trabalho mostra que é viável interpor, entre o desenvolvedor e o provedor de IA, uma camada local que reduz a exposição de credenciais a um custo de análise muito baixo, sem alterar a ferramenta nem o provedor. Mostra também, e este é um resultado tão relevante quanto o primeiro, que a confiança nessa camada depende de avaliação em condições reais e de testes que não sejam escritos apenas para confirmar o que o código já faz. Nesse sentido, o GalileuCLI deve ser lido como uma base verificada em seu núcleo e um ponto de partida para a avaliação mais ampla descrita acima.

Referências

ATRASSI, H. E.; IDRISSI, Y. E.; BENKAOUZ, Y. Codewisp: Ast guided retrieval augmented generation for code generation and completion. In: *2025 12th International Conference on Wireless Networks and Mobile Communications (WINCOM)*. [S.l.: s.n.], 2025. p. 1–6. ISSN 2769-9994. 22, 27

KHAN, A.; RAJAGOPALAN, S. Ai-powered text screening: A generative approach to identifying sensitive information. In: *2025 11th International Conference on Computer and Communications (ICCC)*. [S.l.: s.n.], 2025. p. 438–446. ISSN 2837-7109. 23, 26, 27

PAN, Q.; WU, J. Selective privacy-preserving federated learning for large language model fine-tuning. In: *2025 International Wireless Communications and Mobile Computing (IWCMC)*. [S.l.: s.n.], 2025. p. 1626–1631. ISSN 2376-6506. 24, 25, 26, 27, 38, 76

RAHMAN, M. N. et al. Secret breach detection in source code with large language models. In: *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. [S.l.: s.n.], 2025. p. 207–217. 23, 24, 26, 27, 56, 76, 82

SAKAPERTANA, A. S. et al. Advancing data privacy in rest apis: A comparative study of machine learning techniques for pii detection. In: *2025 International Conference on Smart Computing, IoT and Machine Learning (SIML)*. [S.l.: s.n.], 2025. p. 1–6. 23, 26, 27, 76

TSEBAK, M.; BANAKH, R.; PISKOZUB, A. Enhancing the efficiency of secret detection in version control systems using machine learning methods. In: *2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST)*. [S.l.: s.n.], 2025. p. 1–8. 23, 24, 25, 26, 27, 34, 56, 76, 82

XU, P. et al. Privacy-preserving large language model in terms of secure computing: A survey. In: *2025 2nd International Conference on Algorithms, Software Engineering and Network Security (ASENS)*. [S.l.: s.n.], 2025. p. 286–294. 22, 25, 26, 27, 76

ZHANG, X. et al. Privacyasst: Safeguarding user privacy in tool-using large language model agents. *IEEE Transactions on Dependable and Secure Computing*, v. 21, n. 6, p. 5242–5258, Nov 2024. ISSN 1941-0018. 22, 24, 26, 27

A Apêndice